B. K. BIRLA COLLEGE, KALYAN

# DATA ENGINEERING

## PRACTICAL JOURNAL

| | |
|---|---|
| **Name** | Aayushman Rajeev Verma |
| **Roll No.** | 57 |
| **Student ID** | 5176202 |
| **Subject** | Data Engineering |
| **Practicals** | 1 to 10 |

Practical 10 (Mini Project) was completed as a group with Vedant Vinod Rakshe (Roll No. 37, Student ID 5176220).

# INDEX

Data Engineering Practical Journal | Aayushman Rajeev Verma | Roll No. 57


| Sr. No. | Title of Practical | Page No. |
|---|---|---|
| 1 | Parsing Text, CSV, HTML, XML & JSON, Binary Files, Regular Expressions and SQL CRUD | 2 |
| 2 | ETL Process in Power BI (Extract, Transform and Load) | 10 |
| 3 | MongoDB Operations | 14 |
| 4 | Data Preprocessing, Noise Elimination, Feature Selection and Exploratory Data Analysis | 17 |
| 5 | Extracting Data from APIs and Flat Files using Python | 23 |
| 6 | Installing Apache Airflow and Creating a Scheduled DAG | 26 |
| 7 | ETL Pipelines: CSV and JSON Sources, Invalid Records, Validation and Incremental Loading | 31 |
| 8 | Distributed Data Processing with PySpark DataFrames | 37 |
| 9 | End-to-End Data Pipelines: E-Commerce ETL with Historical and Newly Arriving Data | 42 |
| 10 | Mini Project: Retail Sales Data Warehouse - End-to-End Data Engineering Solution | 48 |

# Practical No. 1: Parsing Text, CSV, HTML, XML & JSON, Binary Files, Regular Expressions and SQL CRUD

**Aim:** To write Python programs that read text, CSV, HTML, XML and JSON documents, extract data from them and check the data for missing values, duplicates and out-of-range (anomalous) values; to read and write binary files; to search, split and replace text using regular expressions; and to design a relational database and perform CRUD operations with SQL.

## Step 1: Create the Employee Data Files (Text, CSV, HTML, XML, JSON)

**Code:**

```
# Name: Ayushman Rajeev Verma
# Practical 1 - Setup: one employee data set stored in five different file formats
print("Name: Ayushman Rajeev Verma")
import json

EMP = [  # (emp_id, name, dept, salary, age)  -> contains deliberate data problems
    ("E101", "Aarav Mehta", "IT", 65000, 28),
    ("E102", "Isha Nair", "HR", 48000, 31),
    ("E103", "Kabir Singh", "IT", -52000, 45),        # negative salary
    ("E104", "Meera Joshi", "", 39000, 26),           # missing department
    ("E105", "Nikhil Rao", "Finance", 72000, 150),    # impossible age
    ("E102", "Isha Nair", "HR", 48000, 31),           # duplicate record
    ("E106", "Pooja Desai", "Sales", None, 35),       # missing salary
    ("E107", "Sameer Khan", "Sales", 41000, 17),      # below working age
]
show = lambda v: "" if v is None else str(v)

with open("employees.txt", "w") as f:                 # pipe-delimited text
    f.write("# Employee export | delimiter = |\n")
    for rec in EMP:
        f.write("|".join(show(v) for v in rec) + "\n")

with open("employees.csv", "w") as f:                 # comma separated
    f.write("emp_id,name,dept,salary,age\n")
    for rec in EMP:
        f.write(",".join(show(v) for v in rec) + "\n")

rows = "".join("<tr>" + "".join(f"<td>{show(v)}</td>" for v in rec) + "</tr>\n" for rec in EMP)
with open("employees.html", "w") as f:                # HTML page with a table
    f.write("<html><body><h1>Employee Directory</h1>\n<table border='1'>\n"
            "<tr><th>emp_id</th><th>name</th><th>dept</th><th>salary</th><th>age</th></tr>\n" + rows +
"</table></body></html>")

with open("employees.xml", "w") as f:                 # XML where the data sits in attributes
    f.write("<company>\n")
    for e, n, d, s, a in EMP:
        f.write(f'  <employee emp_id="{e}" name="{n}" dept="{d}" salary="{show(s)}" age="{a}"/>\n')
    f.write("</company>")

records = [dict(zip(["emp_id", "name", "dept", "salary", "age"], rec)) for rec in EMP]
for r in records:
    r["skills"] = ["python", "sql"] if r["dept"] == "IT" else ["excel"]
with open("employees.json", "w") as f:
    json.dump({"company": "Nova Tech", "employees": records}, f, indent=2)

print("Created: employees.txt, employees.csv, employees.html, employees.xml, employees.json")
```

**Output:**

```
Name: Ayushman Rajeev Verma
Created: employees.txt, employees.csv, employees.html, employees.xml, employees.json
```

## Step 2: A Reusable Data Quality Checker Class

One class is used for every file format. It converts empty values to missing values, fixes data types and reports missing values, duplicate rows and values outside the allowed range (salary above 0 and age between 18 and 65).

**Code:**

```
# Name: Ayushman Rajeev Verma
import pandas as pd
```

```
import numpy as np

class DataQualityChecker:
    """Reusable quality checker: missing values, duplicates and out-of-range values."""
    RULES = {"salary": (1, 1_000_000), "age": (18, 65)}      # allowed (min, max)

    def __init__(self, df, source):
        self.source = source
        self.df = df.replace("", np.nan)                      # treat empty strings as missing
        for col in ("salary", "age"):
            self.df[col] = pd.to_numeric(self.df[col], errors="coerce")

    def missing(self):
        return self.df.isna().sum()

    def duplicates(self):
        return self.df[self.df.duplicated(keep="first")]

    def anomalies(self):
        found = []
        for col, (lo, hi) in self.RULES.items():
            bad = self.df[(self.df[col] < lo) | (self.df[col] > hi)]
            found += [(r.emp_id, col, r[col]) for _, r in bad.iterrows()]
        return found

    def report(self):
        print(f"##### QUALITY REPORT : {self.source} #####")
        print(self.df.to_string(index=False), "\n")
        miss = self.missing()
        print("Missing values :", {c: int(v) for c, v in miss.items() if v})
        dup = self.duplicates()
        print("Duplicate rows :", len(dup), list(dup["emp_id"]))
        print("Out-of-range   :")
        for emp, col, val in self.anomalies():
            print(f"   {emp} -> {col} = {val:g}  (allowed {self.RULES[col][0]} to {self.RULES[col][1]})")
        return self.df
```

## Step 3: Parse a Text File (pipe delimited)

### Code:

```
print("Name: Ayushman Rajeev Verma")
def load_txt(path):
    rows = []
    with open(path) as fh:
        for line in fh:
            if line.startswith("#") or not line.strip():
                continue
            rows.append(line.rstrip("\n").split("|"))
    return pd.DataFrame(rows, columns=["emp_id", "name", "dept", "salary", "age"])

DataQualityChecker(load_txt("employees.txt"), "TEXT (pipe delimited)").report()
```

### Output:

```
Name: Ayushman Rajeev Verma
##### QUALITY REPORT : TEXT (pipe delimited) #####
emp_id        name    dept   salary  age
  E101 Aarav Mehta      IT  65000.0   28
  E102   Isha Nair      HR  48000.0   31
  E103 Kabir Singh      IT -52000.0   45
  E104 Meera Joshi     NaN  39000.0   26
  E105  Nikhil Rao Finance  72000.0  150
  E102   Isha Nair      HR  48000.0   31
  E106 Pooja Desai   Sales      NaN   35
  E107 Sameer Khan   Sales  41000.0   17

Missing values : {'dept': 1, 'salary': 1}
Duplicate rows : 1 ['E102']
Out-of-range   :
   E103 -> salary = -52000  (allowed 1 to 1000000)
   E105 -> age = 150  (allowed 18 to 65)
   E107 -> age = 17  (allowed 18 to 65)
```

## Step 4: Parse a CSV File

### Code:

```
print("Name: Ayushman Rajeev Verma")
df_csv = pd.read_csv("employees.csv", dtype=str, keep_default_na=False)
DataQualityChecker(df_csv, "CSV file").report()
```

### Output:

```
Name: Ayushman Rajeev Verma
##### QUALITY REPORT : CSV file #####
emp_id        name    dept   salary  age
  E101 Aarav Mehta      IT  65000.0   28
  E102   Isha Nair      HR  48000.0   31
  E103 Kabir Singh      IT -52000.0   45
  E104 Meera Joshi     NaN  39000.0   26
  E105  Nikhil Rao Finance  72000.0  150
  E102   Isha Nair      HR  48000.0   31
  E106 Pooja Desai   Sales      NaN   35
  E107 Sameer Khan   Sales  41000.0   17

Missing values : {'dept': 1, 'salary': 1}
Duplicate rows : 1 ['E102']
Out-of-range   :
   E103 -> salary = -52000  (allowed 1 to 1000000)
   E105 -> age = 150  (allowed 18 to 65)
   E107 -> age = 17  (allowed 18 to 65)
```

## Step 5: Parse an HTML Document

### Code:

```
print("Name: Ayushman Rajeev Verma")
import io
from lxml import html as lh

page = open("employees.html").read()
tree = lh.fromstring(page)
print("Page heading :", tree.xpath("//h1/text()")[0])
print("Table rows   :", len(tree.xpath("//table//tr")) - 1, "\n")

df_html = pd.read_html(io.StringIO(page), keep_default_na=False)[0].astype(str)
DataQualityChecker(df_html, "HTML table").report()
```

### Output:

```
Name: Ayushman Rajeev Verma
Page heading : Employee Directory
Table rows   : 8

##### QUALITY REPORT : HTML table #####
emp_id        name    dept   salary  age
  E101 Aarav Mehta      IT  65000.0   28
  E102   Isha Nair      HR  48000.0   31
  E103 Kabir Singh      IT -52000.0   45
  E104 Meera Joshi     NaN  39000.0   26
  E105  Nikhil Rao Finance  72000.0  150
  E102   Isha Nair      HR  48000.0   31
  E106 Pooja Desai   Sales      NaN   35
  E107 Sameer Khan   Sales  41000.0   17

Missing values : {'dept': 1, 'salary': 1}
Duplicate rows : 1 ['E102']
Out-of-range   :
   E103 -> salary = -52000  (allowed 1 to 1000000)
   E105 -> age = 150  (allowed 18 to 65)
   E107 -> age = 17  (allowed 18 to 65)
```

## Step 6: Parse an XML Document

### Code:

```
print("Name: Ayushman Rajeev Verma")
import xml.etree.ElementTree as ET
```

```
root = ET.parse("employees.xml").getroot()
df_xml = pd.DataFrame([emp.attrib for emp in root.iter("employee")])
print("Root tag:", root.tag, "| employee elements:", len(df_xml), "\n")
DataQualityChecker(df_xml, "XML (attributes)").report()
```

**Output:**

```
Name: Ayushman Rajeev Verma
Root tag: company | employee elements: 8

##### QUALITY REPORT : XML (attributes) #####
emp_id        name    dept   salary  age
  E101 Aarav Mehta      IT  65000.0   28
  E102   Isha Nair      HR  48000.0   31
  E103 Kabir Singh      IT -52000.0   45
  E104 Meera Joshi     NaN  39000.0   26
  E105  Nikhil Rao Finance  72000.0  150
  E102   Isha Nair      HR  48000.0   31
  E106 Pooja Desai   Sales      NaN   35
  E107 Sameer Khan   Sales  41000.0   17

Missing values : {'dept': 1, 'salary': 1}
Duplicate rows : 1 ['E102']
Out-of-range   :
   E103 -> salary = -52000  (allowed 1 to 1000000)
   E105 -> age = 150  (allowed 18 to 65)
   E107 -> age = 17  (allowed 18 to 65)
```

## Step 7: Parse a JSON Document

**Code:**

```
print("Name: Ayushman Rajeev Verma")
import json

doc = json.load(open("employees.json"))
print("Company:", doc["company"], "| records:", len(doc["employees"]))
df_json = pd.DataFrame(doc["employees"])
df_json["skill_count"] = df_json["skills"].apply(len)       # derived column from a nested list
print("Skill counts:", dict(zip(df_json["emp_id"], df_json["skill_count"])), "\n")
DataQualityChecker(df_json.drop(columns=["skills", "skill_count"]), "JSON document").report()
```

**Output:**

```
Name: Ayushman Rajeev Verma
Company: Nova Tech | records: 8
Skill counts: {'E101': 2, 'E102': 1, 'E103': 2, 'E104': 1, 'E105': 1, 'E106': 1, 'E107': 1}

##### QUALITY REPORT : JSON document #####
emp_id        name    dept   salary  age
  E101 Aarav Mehta      IT  65000.0   28
  E102   Isha Nair      HR  48000.0   31
  E103 Kabir Singh      IT -52000.0   45
  E104 Meera Joshi     NaN  39000.0   26
  E105  Nikhil Rao Finance  72000.0  150
  E102   Isha Nair      HR  48000.0   31
  E106 Pooja Desai   Sales      NaN   35
  E107 Sameer Khan   Sales  41000.0   17

Missing values : {'dept': 1, 'salary': 1}
Duplicate rows : 1 ['E102']
Out-of-range   :
   E103 -> salary = -52000  (allowed 1 to 1000000)
   E105 -> age = 150  (allowed 18 to 65)
   E107 -> age = 17  (allowed 18 to 65)
```

## Step 8: Binary Files - Custom File Format with Header and Records

**Code:**

```
print("Name: Ayushman Rajeev Verma")
import struct, zlib, os

MAGIC, VERSION = b"EMPD", 1
```

```
HEADER = struct.Struct("<4sBI")            # magic, version, record count
RECORD = struct.Struct("<4sfB")            # emp code, salary, age

staff = [(b"E101", 65000.0, 28), (b"E102", 48000.0, 31), (b"E106", 39500.5, 35)]
with open("staff.bin", "wb") as f:                          # ---- WRITE
    f.write(HEADER.pack(MAGIC, VERSION, len(staff)))
    for rec in staff:
        f.write(RECORD.pack(*rec))
print("File size:", os.path.getsize("staff.bin"), "bytes  =", HEADER.size, "header +", len(staff), "x", RECORD.size)

data = open("staff.bin", "rb").read()                       # ---- READ (whole file as bytes)
print("First 16 bytes:", data[:16].hex(" "))
magic, ver, count = HEADER.unpack_from(data, 0)
print("Header ->", magic.decode(), "| version", ver, "| records", count)
for i in range(count):
    code, sal, age = RECORD.unpack_from(data, HEADER.size + i * RECORD.size)
    print(f"  record {i+1}: {code.decode()}  salary={sal:.1f}  age={age}")

buf = bytearray(data)                                       # ---- MODIFY in memory, then save
RECORD.pack_into(buf, HEADER.size + RECORD.size, b"E102", 52000.0, 32)
open("staff_updated.bin", "wb").write(buf)
print("Updated record 2 ->", RECORD.unpack_from(open("staff_updated.bin", "rb").read(), HEADER.size + RECORD.size))

print("CRC32 original :", hex(zlib.crc32(data)))            # integrity check
print("CRC32 modified :", hex(zlib.crc32(bytes(buf))))
```

**Output:**

```
Name: Ayushman Rajeev Verma
File size: 36 bytes  = 9 header + 3 x 9
First 16 bytes: 45 4d 50 44 01 03 00 00 00 45 31 30 31 00 e8 7d
Header -> EMPD | version 1 | records 3
  record 1: E101  salary=65000.0  age=28
  record 2: E102  salary=48000.0  age=31
  record 3: E106  salary=39500.5  age=35
Updated record 2 -> (b'E102', 52000.0, 32)
CRC32 original : 0xc497c3
CRC32 modified : 0xae62a160
```

## Step 9: Regular Expressions on a Web-Server Log

**Code:**

```
print("Name: Ayushman Rajeev Verma")
import re

log = """2026-03-04 10:15:32 INFO  192.168.1.10 GET /home 200
2026-03-04 10:16:05 ERROR 10.0.0.77 POST /login 500
2026-03-04 10:17:44 WARN  172.16.4.9 GET /reports/q1.pdf 404
2026-03-04 10:19:10 ERROR 192.168.1.10 GET /api/data 503"""

# 1. SEARCH with a compiled pattern and named groups
pattern = re.compile(r"(?P<date>\d{4}-\d{2}-\d{2}) (?P<time>[\d:]{8}) (?P<level>[A-Z]+)\s+(?P<ip>[\d.]+)
(?P<method>GET|POST) (?P<url>\S+) (?P<code>\d{3})")
for m in pattern.finditer(log):
    print(m.group("level").ljust(5), m.group("ip").ljust(13), m.group("url").ljust(16), m.group("code"))

print("Error lines :", len(re.findall(r"^.*ERROR.*$", log, flags=re.M)))
print("Unique IPs  :", sorted(set(re.findall(r"\b(?:\d{1,3}\.){3}\d{1,3}\b", log))))

# 2. SPLIT
first = log.split("\n")[0]
print("Split line  :", re.split(r"\s+", first))
print("Split on / and space:", re.split(r"[/\s]+", "GET /reports/q1.pdf 404"))

# 3. REPLACE (plain text and with a function)
mask = lambda m: ".".join(m.group().split(".")[:2] + ["x", "x"])
print("Masked IPs  :", re.findall(r"\b\d+\.\d+\.x\.x\b", re.sub(r"\b(?:\d{1,3}\.){3}\d{1,3}\b", mask, log)))
print("Levels lower:", re.sub(r"\b(INFO|WARN|ERROR)\b", lambda m: m.group().lower(), first))
print("Dates DD/MM/YYYY:", re.sub(r"(\d{4})-(\d{2})-(\d{2})", r"\3/\2/\1", first)[:10])
```

**Output:**

```
Name: Ayushman Rajeev Verma
INFO  192.168.1.10  /home           200
ERROR 10.0.0.77     /login          500
WARN  172.16.4.9    /reports/q1.pdf  404
ERROR 192.168.1.10  /api/data       503
Error lines : 2
Unique IPs  : ['10.0.0.77', '172.16.4.9', '192.168.1.10']
Split line  : ['2026-03-04', '10:15:32', 'INFO', '192.168.1.10', 'GET', '/home', '200']
Split on / and space: ['GET', 'reports', 'q1.pdf', '404']
Masked IPs  : ['192.168.x.x', '10.0.x.x', '172.16.x.x', '192.168.x.x']
Levels lower: 2026-03-04 10:15:32 info  192.168.1.10 GET /home 200
Dates DD/MM/YYYY: 04/03/2026
```

## Step 10: Relational Database Design (Library System) and CRUD Operations

Three related tables - Book, Member and Issue - are created. Issue links books and members through foreign keys; deleting a book also removes its issue records (ON DELETE CASCADE).

**Code:**

```
print("Name: Ayushman Rajeev Verma")
import sqlite3

con = sqlite3.connect("library.db")
cur = con.cursor()
cur.executescript("""
PRAGMA foreign_keys = ON;
DROP TABLE IF EXISTS Issue; DROP TABLE IF EXISTS Member; DROP TABLE IF EXISTS Book;
CREATE TABLE Book   (book_id INTEGER PRIMARY KEY, title TEXT NOT NULL, author TEXT, price REAL CHECK (price > 0),
copies INTEGER DEFAULT 1);
CREATE TABLE Member (member_id INTEGER PRIMARY KEY, name TEXT NOT NULL, joined TEXT);
CREATE TABLE Issue  (issue_id INTEGER PRIMARY KEY AUTOINCREMENT,
                     book_id INTEGER NOT NULL REFERENCES Book(book_id) ON DELETE CASCADE,
                     member_id INTEGER NOT NULL REFERENCES Member(member_id),
                     issue_date TEXT, returned INTEGER DEFAULT 0);
""")

def table(sql, params=()):
    print(pd.read_sql(sql, con, params=params).to_string(index=False), "\n")

# ---- CREATE
cur.executemany("INSERT INTO Book (title, author, price, copies) VALUES (?,?,?,?)", [
    ("Database Systems", "Elmasri", 650.0, 4), ("Python Basics", "Matthes", 480.0, 6), ("Data Pipelines", "Reis",
720.0, 2)])
cur.executemany("INSERT INTO Member (name, joined) VALUES (?,?)", [("Anaya", "2026-01-10"), ("Veer", "2026-02-01")])
cur.executemany("INSERT INTO Issue (book_id, member_id, issue_date) VALUES (?,?,?)", [(1, 1, "2026-03-01"), (2, 1,
"2026-03-02"), (3, 2, "2026-03-03")])
con.commit()
print("CREATE -> books:"); table("SELECT * FROM Book")

# ---- READ
print("READ -> books issued per member:")
table("""SELECT m.name AS member, COUNT(*) AS books_issued, GROUP_CONCAT(b.title, ' | ') AS titles
         FROM Issue i JOIN Member m USING (member_id) JOIN Book b USING (book_id) GROUP BY m.name""")
print("READ -> books costing more than 500:")
table("SELECT title, price FROM Book WHERE price > ? ORDER BY price DESC", (500,))

# ---- UPDATE
cur.execute("UPDATE Book SET copies = copies - 1 WHERE book_id = 1")
cur.execute("UPDATE Issue SET returned = 1 WHERE issue_id = 2")
con.commit()
print("UPDATE -> stock and returns:")
table("SELECT book_id, title, copies FROM Book")
table("SELECT issue_id, book_id, returned FROM Issue")

# ---- DELETE (cascades to the Issue table)
cur.execute("DELETE FROM Book WHERE book_id = 3")
con.commit()
print("DELETE -> remaining books and issues:")
table("SELECT book_id, title FROM Book")
table("SELECT issue_id, book_id, member_id FROM Issue")
con.close()
```

**Output:**

```
Name: Ayushman Rajeev Verma
CREATE -> books:
 book_id            title  author  price  copies
       1 Database Systems Elmasri  650.0       4
       2    Python Basics Matthes  480.0       6
       3   Data Pipelines    Reis  720.0       2

READ -> books issued per member:
member  books_issued                          titles
 Anaya             2 Database Systems | Python Basics
  Veer             1                   Data Pipelines

READ -> books costing more than 500:
           title  price
  Data Pipelines  720.0
Database Systems  650.0

UPDATE -> stock and returns:
 book_id            title  copies
       1 Database Systems       3
       2    Python Basics       6
       3   Data Pipelines       2

 issue_id  book_id  returned
        1        1         0
        2        2         1
        3        3         0

DELETE -> remaining books and issues:
 book_id            title
       1 Database Systems
       2    Python Basics

 issue_id  book_id  member_id
        1        1          1
        2        2          1
```

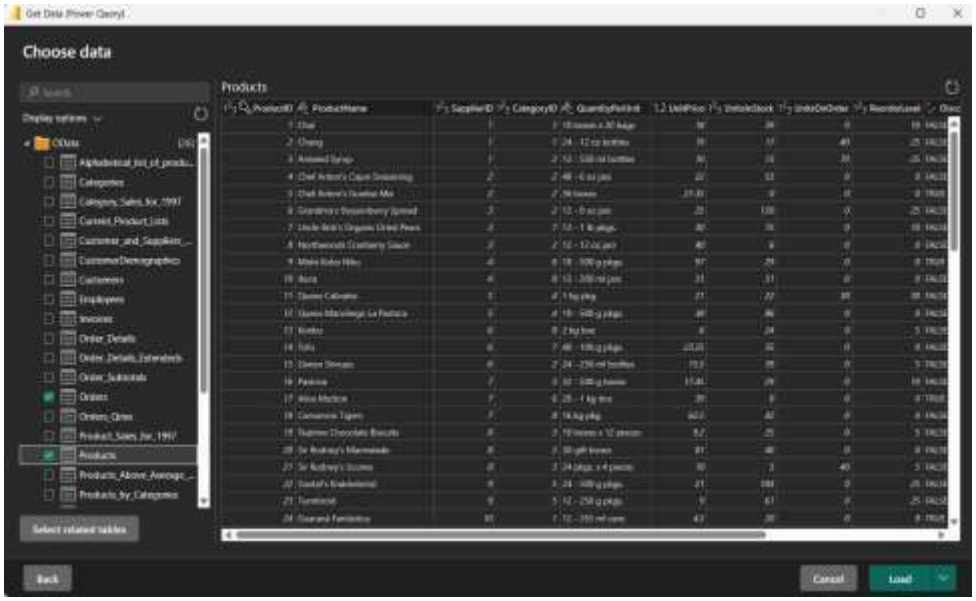
# Practical No. 2: ETL Process in Power BI (Extract, Transform and Load)

**Aim:** To perform the ETL process in Power BI by extracting, transforming, and loading data to prepare it for analysis.

## Step 1: Import Excel Data into Power BI

The Excel workbook containing the Products, Orders, and Order_Details tables was imported into Power BI for further data transformation and analysis.

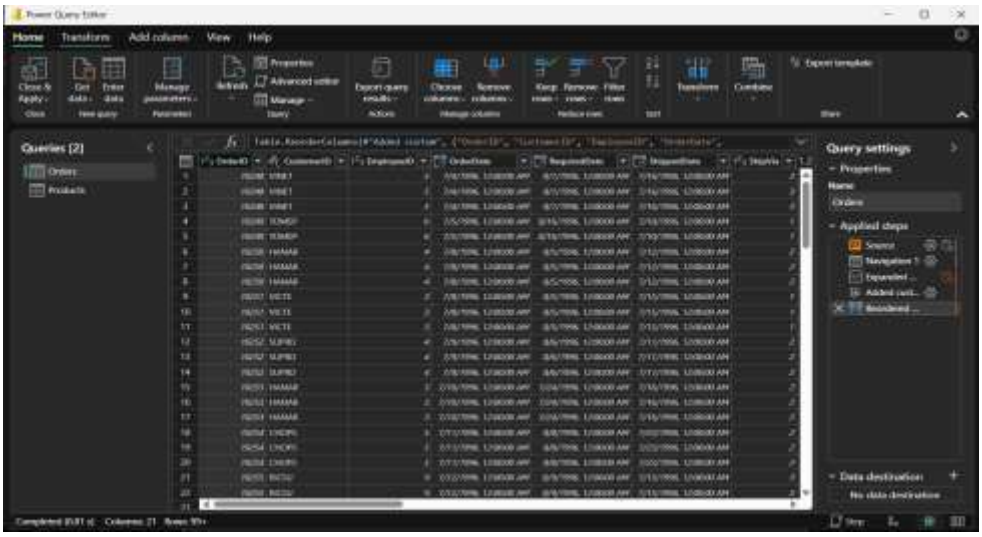
**Output:**




## Step 2: Open Power Query Editor

Power Query Editor was opened to perform the required data transformation operations on the imported tables.

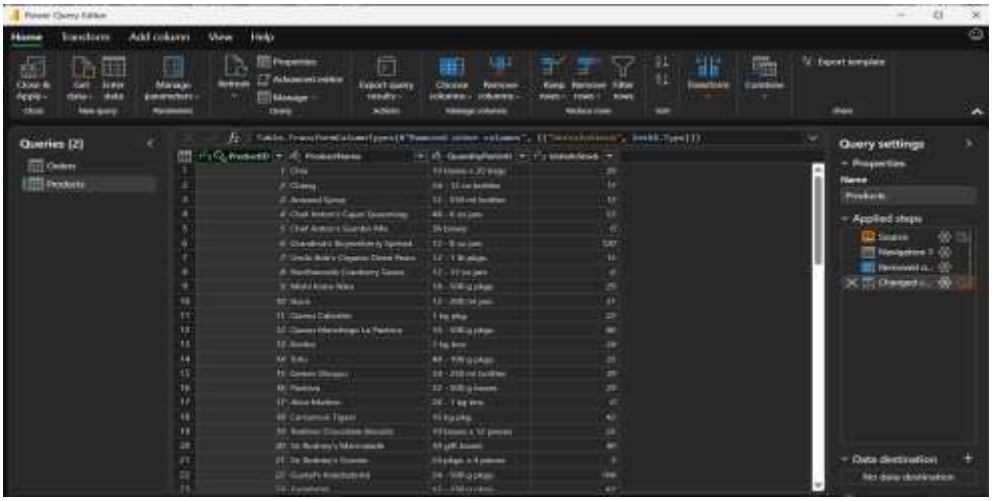
**Output:**




## Step 3: Remove Unnecessary Columns

Unnecessary columns were removed from the Products table to retain only the fields required for further analysis.

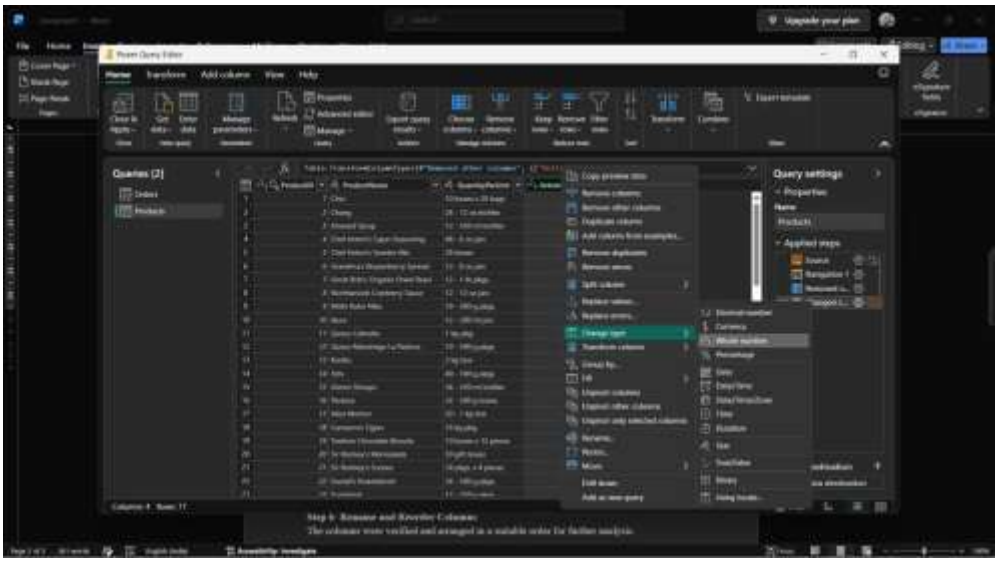
**Output:**



## Step 4: Verify the Data Type of UnitsInStock

The data type of the UnitsInStock column was verified as Whole Number, which is appropriate for representing stock quantities.

**Output:**



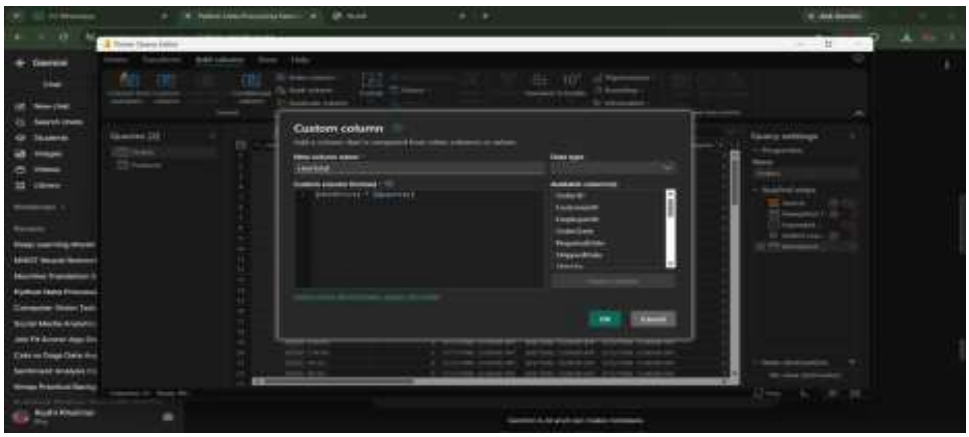
## Step 5: Calculate Line Total

A new LineTotal column was created by multiplying UnitPrice by Quantity for each order detail (Add Column > Custom Column).

**Code:**

```
LineTotal = [UnitPrice] * [Quantity]
```

**Output:**




## Step 6: Rename and Reorder Columns

The columns were verified and arranged in a suitable order for further analysis.

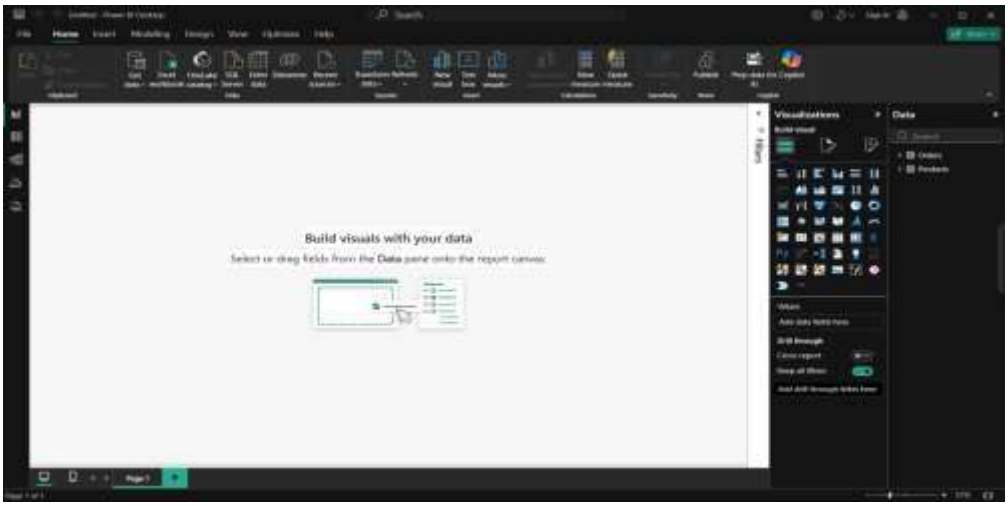
**Output:**



## Step 7: Load the Transformed Data

The transformed data was loaded into Power BI Desktop using the Close & Apply option.

**Output:**




## Step 8: Create Relationships Between Tables

Relationships were created between the related tables using their common key fields to establish the data model.

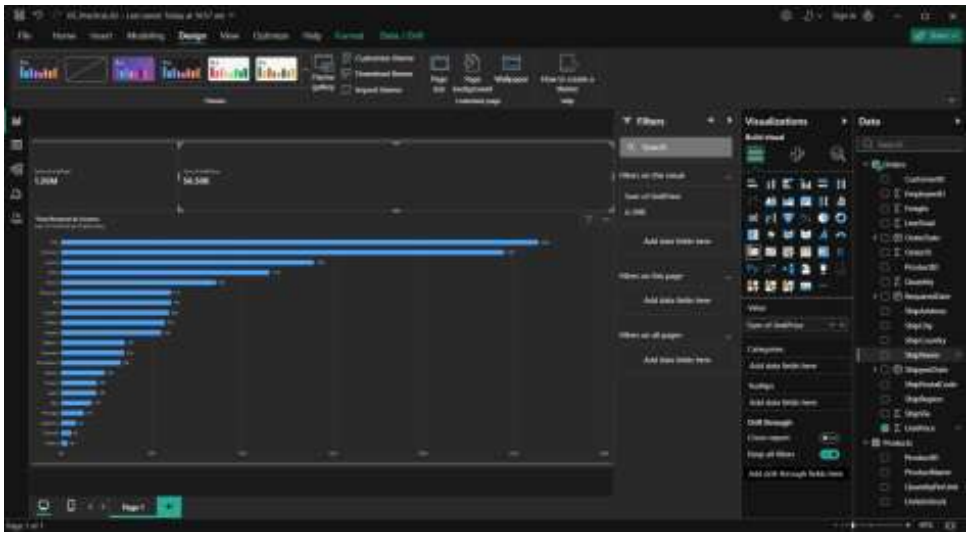
**Output:**



## Step 9: Result

The ETL process was successfully performed in Power BI by extracting data from an Excel workbook, transforming the data using Power Query, creating the LineTotal column, loading the transformed data, and establishing relationships between the tables. A report page with card visuals and a clustered bar chart of Sum of LineTotal by ShipCountry was built on the loaded model.

**Output:**



# Practical No. 3: MongoDB Operations

**Aim:** To perform single and multiple document insertion and retrieval using MongoDB shell (mongosh).

## Step 1: Insert Single Document

**Code:**

```
db.items.insertOne({ name: "laptop", price: 999 });
```

**Output:**

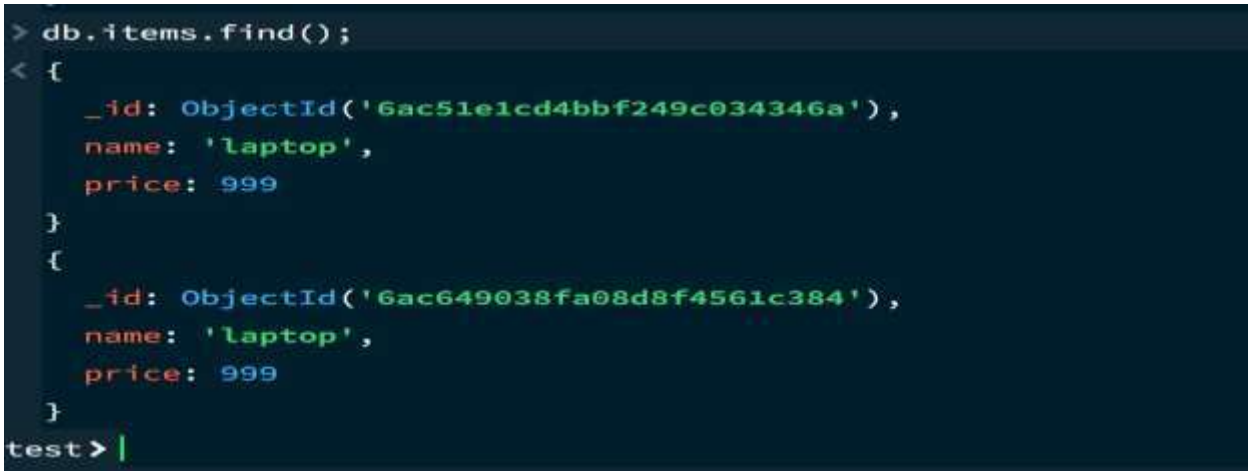



## Step 2: View Inserted Document

**Code:**

```
db.items.find();
```

**Output:**




## Step 3: Insert Multiple Documents

**Code:**

```
db.products.insertMany([
  { name: "phone", price: 500, stock: 10 },
  { name: "tablet", price: 300, stock: 5 },
  { name: "watch", price: 150, stock: 0 }
]);
```

**Output:**

```
> db.products.insertMany([
    { name: "phone", price: 500, stock: 10 },
    { name: "tablet", price: 300, stock: 5 },
    { name: "watch", price: 150, stock: 0 }
  ]);
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6ac6498b8fa08d8f4561c385'),
      '1': ObjectId('6ac6498b8fa08d8f4561c386'),
      '2': ObjectId('6ac6498b8fa08d8f4561c387')
    }
  }
test>
```

## Step 4: View Products in Formatted JSON

**Code:**

```
db.products.find().pretty();
```

**Output:**

```
> db.products.find().pretty();
< {
    _id: ObjectId('6ac51e43d4bbf249c834346b'),
    name: 'phone',
    price: 500,
    stock: 10
  }
  {
    _id: ObjectId('6ac51e43d4bbf249c834346c'),
    name: 'tablet',
    price: 300,
    stock: 5
  }
  {
    _id: ObjectId('6ac51e43d4bbf249c834346d'),
    name: 'watch',
    price: 150,
    stock: 0
  }
  {
    _id: ObjectId('6ac6498b8fa08d8f4561c385'),
    name: 'phone',
    price: 500,
    stock: 10
  }
```

**Output:**

```
  {
    _id: ObjectId('6ac6498b8fa08d8f4561c386'),
    name: 'tablet',
    price: 300,
    stock: 5
  }
  {
    _id: ObjectId('6ac6498b8fa08d8f4561c387'),
    name: 'watch',
    price: 150,
    stock: 0
  }
test>
```

# Practical No. 4: Data Preprocessing, Noise Elimination, Feature Selection and Exploratory Data Analysis

**Aim:** To use Pandas, NumPy, Scikit-learn, Matplotlib and Seaborn to clean a data set (missing values and duplicates), scale it (normalisation and standardisation), remove noise, select the useful features and explore the data with statistics and plots (EDA).

## Step 1: Cleaning, Normalization and Standardization (Part A)

A synthetic housing data set (32 rows) is generated with deliberate problems: missing numbers and text, two duplicate rows, a constant column, a redundant column (price in lakh) and two noisy prices.

**Code:**

```
# Name: Ayushman Rajeev Verma
# PART A : data cleaning and scaling on a housing data set
print("Name: Ayushman Rajeev Verma")
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

rng = np.random.default_rng(7)
n = 30
houses = pd.DataFrame({
    "sqft":      rng.integers(600, 2400, n).astype(float),
    "bedrooms":  rng.integers(1, 5, n).astype(float),
    "age_years": rng.integers(1, 40, n).astype(float),
    "distance_km": np.round(rng.uniform(1, 25, n), 1),
    "city":      rng.choice(["Pune", "Mumbai", "Nashik"], n),
})
houses["price"] = (houses["sqft"] * 4200 - houses["age_years"] * 9000 - houses["distance_km"] * 12000
                   + rng.normal(0, 60000, n)).round(-3)
houses["price_lakh"] = houses["price"] / 100000          # same information as price (redundant)
houses["country_code"] = 91                               # constant column
houses = pd.concat([houses, houses.iloc[[3, 11]]], ignore_index=True)       # 2 duplicate rows
houses.loc[[2, 9, 17], "sqft"] = np.nan                                      # missing numbers
houses.loc[[5, 20], "city"] = None                                           # missing text
houses.loc[[4], "price"] = 45_000_000                                        # extreme price (noise)
houses.loc[[13], "price"] = -250_000                                         # impossible price (noise)

print("Shape:", houses.shape)
print("\nMissing values:\n", houses.isna().sum()[lambda s: s > 0].to_string())
print("\nDuplicate rows:", houses.duplicated().sum())

clean = houses.drop_duplicates().copy()                                      # 1. duplicates
clean["sqft"] = clean["sqft"].fillna(clean["sqft"].median())                 # 2. numeric -> median
clean["city"] = clean["city"].fillna(clean["city"].mode()[0])                #    text -> most common
print("\nAfter cleaning -> rows:", len(clean), "| missing values left:", int(clean.isna().sum().sum()))

cols = ["sqft", "bedrooms", "age_years", "distance_km"]
scaled_mm = pd.DataFrame(MinMaxScaler().fit_transform(clean[cols]), columns=cols)         # 3. normalisation
scaled_z = pd.DataFrame(StandardScaler().fit_transform(clean[cols]), columns=cols)        #    standardisation
print("\nMin-Max scaled (first 4 rows):\n", scaled_mm.head(4).round(3).to_string())
print("\nZ-score scaled (mean, std):\n", pd.DataFrame({"mean": scaled_z.mean().round(3) + 0.0, "std":
scaled_z.std(ddof=0).round(3)}).to_string())
```

**Output:**

```
Name: Ayushman Rajeev Verma
Shape: (32, 8)

Missing values:
 sqft    3
city    2

Duplicate rows: 2

After cleaning -> rows: 30 | missing values left: 0

Min-Max scaled (first 4 rows):
     sqft  bedrooms  age_years  distance_km
0  0.954     0.667      0.263        0.742
1  0.629     0.667      0.500        0.036
2  0.502     0.667      0.368        0.525
3  0.905     1.000      0.237        0.489

Z-score scaled (mean, std):
              mean  std
sqft           0.0  1.0
bedrooms       0.0  1.0
age_years      0.0  1.0
distance_km    0.0  1.0
```

## Step 2: Noise Elimination (Z-score filter and Smoothing) (Part B1)

**Code:**

```
# PART B1 : noise elimination
print("Name: Ayushman Rajeev Verma")
import matplotlib.pyplot as plt

# (a) outliers in price: keep rows whose |z-score| is at most 2.5
price_z = (clean["price"] - clean["price"].mean()) / clean["price"].std()
noisy_rows = clean[price_z.abs() > 2.5]
print("Rows removed as noise (|z| > 2.5):\n", noisy_rows[["sqft", "price"]].to_string())
print("Rows with impossible (non-positive) price:\n", clean[clean["price"] <= 0][["sqft", "price"]].to_string())
no_outliers = clean[price_z.abs() <= 2.5]
data = no_outliers[no_outliers["price"] > 0].reset_index(drop=True)        # impossible values removed too
print("Rows before:", len(clean), "| after noise removal:", len(data))

# (b) smoothing a noisy sensor signal with a moving average
t = np.arange(40)
signal = np.sin(t / 5) * 10 + 50 + rng.normal(0, 2.5, 40)
signal[[8, 25]] += 25                                          # two spikes
smooth = pd.Series(signal).rolling(window=5, center=True, min_periods=1).median()
print("\nSignal std before smoothing:", round(signal.std(), 2), "| after:", round(smooth.std(), 2))

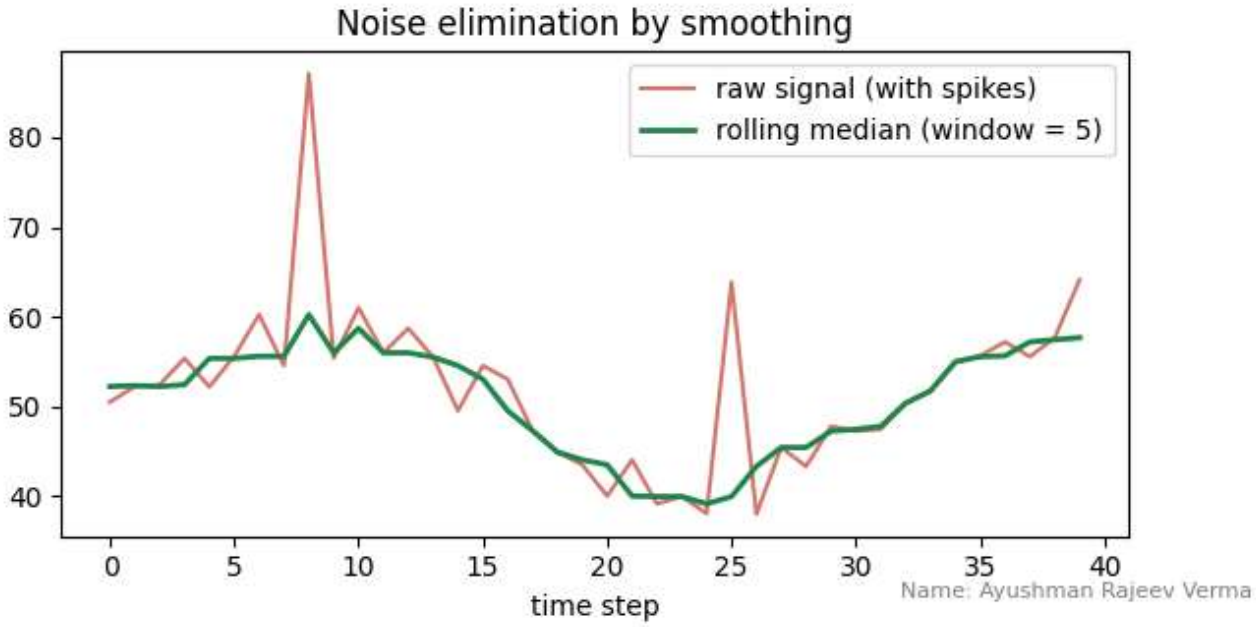
plt.figure(figsize=(7, 3.2))
plt.plot(t, signal, color="#c0392b", alpha=0.7, label="raw signal (with spikes)")
plt.plot(t, smooth, color="#1e8449", lw=2, label="rolling median (window = 5)")
plt.legend(); plt.title("Noise elimination by smoothing"); plt.xlabel("time step")
plt.gcf().text(0.99, 0.005, "Name: Ayushman Rajeev Verma", ha="right", va="bottom", fontsize=8, color="gray")
plt.savefig("p4_noise.png", dpi=110, bbox_inches="tight")
plt.show()
```

**Output:**

```
Name: Ayushman Rajeev Verma
Rows removed as noise (|z| > 2.5):
      sqft       price
4  1640.0  45000000.0
Rows with impossible (non-positive) price:
      sqft     price
13  609.0 -250000.0
Rows before: 30 | after noise removal: 28

Signal std before smoothing: 8.85 | after: 6.13
```

**Output:**




## Step 3: Feature Selection (Variance, Correlation and F-test) (Part B2)

**Code:**

```
# PART B2 : feature selection
print("Name: Ayushman Rajeev Verma")
from sklearn.feature_selection import VarianceThreshold, SelectKBest, f_regression

features = ["sqft", "bedrooms", "age_years", "distance_km", "price_lakh", "country_code"]
X, y = data[features], data["price"]

# step 1 - remove constant columns
vt = VarianceThreshold(threshold=0.0).fit(X)
print("Constant features removed:", [c for c, keep in zip(features, vt.get_support()) if not keep])
X = X.loc[:, vt.get_support()]

# step 2 - remove one of every pair of highly correlated features (|r| > 0.95)
corr = X.corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
redundant = [c for c in upper.columns if (upper[c] > 0.95).any()]
print("Redundant (correlated) features removed:", redundant)
X = X.drop(columns=redundant)

# step 3 - keep the 3 features most related to price (ANOVA F-test)
sk = SelectKBest(f_regression, k=3).fit(X, y)
scores = pd.Series(sk.scores_, index=X.columns).sort_values(ascending=False)
print("\nF-scores:\n", scores.round(1).to_string())
selected = list(X.columns[sk.get_support()])
print("\nSelected features:", selected)
```

**Output:**

```
Name: Ayushman Rajeev Verma
Constant features removed: ['country_code']
Redundant (correlated) features removed: ['price_lakh']

F-scores:
 sqft          335.8
age_years        5.8
bedrooms         1.1
distance_km      0.4

Selected features: ['sqft', 'bedrooms', 'age_years']
```

## Step 4: Exploratory Data Analysis (Part C)

### Code:

```
# PART C : exploratory data analysis (EDA)
print("Name: Ayushman Rajeev Verma")
import seaborn as sns
eda = data[selected + ["price", "city"]]

print("Dataset information:")
eda.info()
print("\nStatistical summary:\n", eda.describe().round(1).to_string())
print("\nCity-wise average price:\n", eda.groupby("city")["price"].agg(["count", "mean"]).round(0).to_string())
print("\nCorrelation with price:\n", eda.drop(columns="city").corr()["price"].round(3).to_string())

fig, ax = plt.subplots(2, 2, figsize=(10, 7))
sns.histplot(eda["price"], bins=10, color="#2e86c1", ax=ax[0, 0]).set_title("Histogram of price")
sns.boxplot(data=eda, x="city", y="price", palette="Set2", ax=ax[0, 1]).set_title("Price by city")
sns.heatmap(eda.drop(columns="city").corr(), annot=True, fmt=".2f", cmap="YlGnBu", ax=ax[1, 0]); ax[1,
0].set_title("Correlation heatmap")
sns.scatterplot(data=eda, x="sqft", y="price", hue="city", ax=ax[1, 1]); ax[1, 1].set_title("Area vs price")
plt.tight_layout()
plt.gcf().text(0.99, 0.005, "Name: Ayushman Rajeev Verma", ha="right", va="bottom", fontsize=8, color="gray")
plt.savefig("p4_eda_overview.png", dpi=110, bbox_inches="tight")
plt.show()
```

### Output:

```
Name: Ayushman Rajeev Verma
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 28 entries, 0 to 27
Data columns (total 5 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   sqft       28 non-null     float64
 1   bedrooms   28 non-null     float64
 2   age_years  28 non-null     float64
 3   price      28 non-null     float64
 4   city       28 non-null     object
dtypes: float64(4), object(1)
memory usage: 1.2+ KB

Statistical summary:
          sqft  bedrooms  age_years      price
count    28.0      28.0       28.0       28.0
mean   1570.2       2.8       20.5  6336250.0
std     501.4       1.1       12.0  2210387.9
min     699.0       1.0        1.0  2491000.0
25%    1137.0       2.0        9.5  4299000.0
50%    1499.0       3.0       20.5  6092500.0
75%    2071.2       4.0       30.0  8426750.0
max    2382.0       4.0       38.0  9510000.0

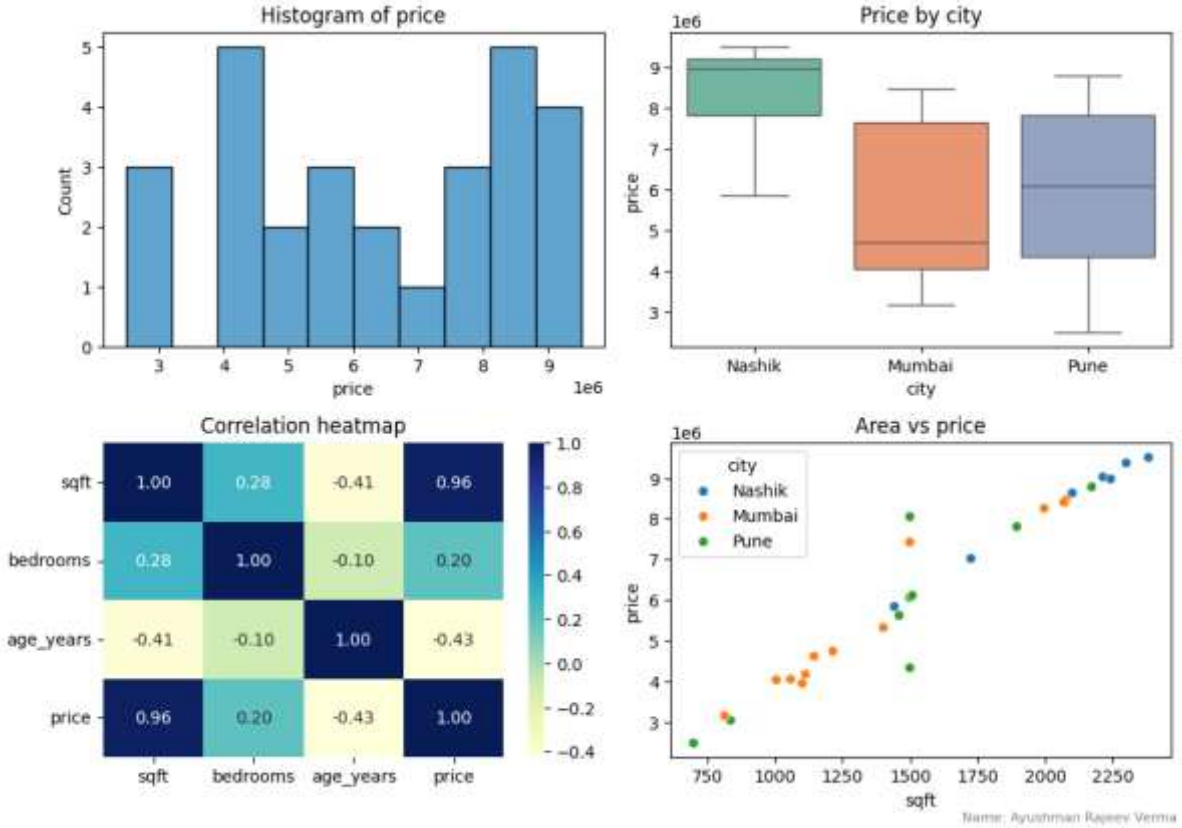
City-wise average price:
         count       mean
city
Mumbai     12  5556583.0
Nashik      7  8342571.0
Pune        9  5815333.0

Correlation with price:
 sqft         0.963
bedrooms      0.200
age_years    -0.426
price         1.000
/tmp/ipykernel_632/3934717536.py:14: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` va
riable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(data=eda, x="city", y="price", palette="Set2", ax=ax[0, 1]).set_title("Price by city")
```

**Output:**


Histogram of price
Count
price
1e6
Price by city
1e6
price
Nashik
Mumbai
Pune
city
Correlation heatmap
sqft
bedrooms
age_years
price
1.00
0.28
-0.41
0.96
0.28
1.00
-0.10
0.20
-0.41
-0.10
1.00
-0.43
0.96
0.20
-0.43
1.00
Area vs price
1e6
city
Nashik
Mumbai
Pune
price
sqft
Name: Ayushman Rajeev Verma

# Practical No. 5: Extracting Data from APIs and Flat Files using Python

**Aim:** To write a Python program (extract_pipeline.py) that collects task data from a public REST API with retry handling, reads a flat CSV file, merges and transforms both sources and saves a clean report as CSV and JSON.

## Step 1: API + CSV Extraction and Transformation (extract_pipeline.py)

The complete program is provided as practical_05_extract_pipeline.py (it is copied as extract_pipeline.py for the Airflow DAG of Practical 6). API used: https://jsonplaceholder.typicode.com/todos?_limit=10

**Code:**

```
# Name: Ayushman Rajeev Verma
# extract_pipeline.py : REST API (JSON) + flat file (CSV) -> one clean report file
print("Name: Ayushman Rajeev Verma")
import json, time
import pandas as pd
import requests

API_URL = "https://jsonplaceholder.typicode.com/todos?_limit=10"
CACHED_SAMPLE = [      # cached sample of the same API response, used only when the API is unreachable
    {"userId": 1, "id": 1, "title": "delectus aut autem", "completed": False},
    {"userId": 1, "id": 2, "title": "quis ut nam facilis et officia qui", "completed": False},
    {"userId": 1, "id": 3, "title": "fugiat veniam minus", "completed": False},
    {"userId": 1, "id": 4, "title": "et porro tempora", "completed": True},
    {"userId": 1, "id": 5, "title": "laboriosam mollitia et enim quasi adipisci quia provident illum", "completed":
False},
    {"userId": 1, "id": 6, "title": "qui ullam ratione quibusdam voluptatem quia omnis", "completed": False},
    {"userId": 1, "id": 7, "title": "illo expedita consequatur quia in", "completed": False},
    {"userId": 1, "id": 8, "title": "quo adipisci enim quam ut ab", "completed": True},
    {"userId": 1, "id": 9, "title": "molestiae perspiciatis ipsa", "completed": False},
    {"userId": 1, "id": 10, "title": "illo est ratione doloremque quia maiores aut", "completed": True},
]

def fetch_json(url, retries=3, wait=1):
    """GET with retries; returns (data, source_label)."""
    for attempt in range(1, retries + 1):
        try:
            r = requests.get(url, timeout=8)
            r.raise_for_status()
            return r.json(), "live API"
        except requests.RequestException as err:
            print(f"  attempt {attempt}/{retries} failed: {type(err).__name__}")
            time.sleep(wait)
    return CACHED_SAMPLE, "cached sample (API not reachable)"

def read_categories(path):
    pd.DataFrame({
        "id": range(1, 11),
        "category": ["work", "home", "work", "study", "home", "study", "work", "health", "home", "health"],
        "due_in_days": [2, 5, 1, 7, 3, 10, 2, 4, 6, 8],
    }).to_csv(path, index=False)            # create the flat file used in this lab
    return pd.read_csv(path)

def build_report(api_rows, cat_df):
    todos = pd.json_normalize(api_rows)
    todos["status"] = todos["completed"].map({True: "Done", False: "Pending"})
    todos["title_words"] = todos["title"].str.split().str.len()
    todos["title"] = todos["title"].str.capitalize()
    report = todos.merge(cat_df, on="id", how="left", validate="one_to_one")
    return report[["id", "userId", "title", "category", "due_in_days", "title_words", "status"]]

if __name__ == "__main__":
    print("Step 1: calling the API ...")
    rows, source = fetch_json(API_URL, retries=2, wait=0)
    print("Data source:", source, "| records:", len(rows))

    print("\nStep 2: reading the CSV file")
    categories = read_categories("task_categories.csv")
    print(categories.head(3).to_string(index=False))

    print("\nStep 3: merging and transforming")
    report = build_report(rows, categories)
    pd.set_option("display.width", 130, "display.max_colwidth", 38)
    print(report.to_string(index=False))
```

```
    print("\nStep 4: summary by category")
    print(report.groupby("category").agg(tasks=("id", "count"), avg_due_days=("due_in_days", "mean"),
                                         done=("status", lambda s: (s == "Done").sum())).round(1).to_string())
    report.to_csv("todo_report.csv", index=False)
    report.to_json("todo_report.json", orient="records", indent=1)
    print("\nSaved: todo_report.csv and todo_report.json")
```

**Output:**

```
Name: Ayushman Rajeev Verma
Step 1: calling the API ...
Data source: live API | records: 10

Step 2: reading the CSV file
 id category  due_in_days
  1     work            2
  2     home            5
  3     work            1

Step 3: merging and transforming
 id  userId                                                        title category  due_in_days  title_
words  status
  1       1                                          Delectus aut autem     work            2
    3 Pending
  2       1                               Quis ut nam facilis et officia qui     home            5
    7 Pending
  3       1                                         Fugiat veniam minus     work            1
    3 Pending
  4       1                                            Et porro tempora    study            7
    3    Done
  5       1 Laboriosam mollitia et enim quasi adipisci quia provident illum     home            3
    9 Pending
  6       1               Qui ullam ratione quibusdam voluptatem quia omnis    study           10
    7 Pending
  7       1                                  Illo expedita consequatur quia in     work            2
    5 Pending
  8       1                                       Quo adipisci enim quam ut ab   health            4
    6    Done
  9       1                                        Molestiae perspiciatis ipsa     home            6
    3 Pending
 10       1                     Illo est ratione doloremque quia maiores aut   health            8
    7    Done

Step 4: summary by category
          tasks  avg_due_days  done
category
health        2           6.0     2
home          3           4.7     0
study         2           8.5     1
work          3           1.7     0

Saved: todo_report.csv and todo_report.json
```

## Step 2: Verify the Saved Report

**Code:**

```
print("Name: Ayushman Rajeev Verma")
import pandas as pd
check = pd.read_csv("todo_report.csv")
print("Rows:", len(check), "| Columns:", list(check.columns))
print("Duplicate ids :", check["id"].duplicated().sum())
print("Missing values:", int(check.isna().sum().sum()))
print("Pending tasks :", (check["status"] == "Pending").sum(), "| Done:", (check["status"] == "Done").sum())
```

**Output:**

```
Name: Ayushman Rajeev Verma
Rows: 10 | Columns: ['id', 'userId', 'title', 'category', 'due_in_days', 'title_words', 'status']
Duplicate ids : 0
Missing values: 0
Pending tasks : 7 | Done: 3
```

# Practical No. 6: Installing Apache Airflow and Creating a Scheduled DAG

**Aim:** To set up Apache Airflow and write a DAG that runs every day at 06:00 and executes five dependent tasks: a start marker, the extraction program of Practical 5, a Python quality check, a success notification and an end marker.

Airflow runs on Linux / macOS / WSL2 / Google Colab. This notebook installs Airflow, writes the extraction program (Practical 5) and the DAG file into `~/airflow`, validates the DAG, runs its quality check and draws the task graph.

## Step 1: Install Apache Airflow

**Code:**

```
%pip install -q apache-airflow
```

**Output:**

```
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed
. This behaviour is the source of the following dependency conflicts.
google-adk 2.7.1 requires opentelemetry-api<=1.42.1,>=1.39, but you have opentelemetry-api 1.45.1 which i
s incompatible.
google-adk 2.7.1 requires opentelemetry-sdk<=1.42.1,>=1.39, but you have opentelemetry-sdk 1.45.1 which i
s incompatible.
```

**Code:**

```
!airflow db migrate
```

**Output:**

```
2026-10-09T16:32:07.718308Z [info     ] Performing upgrade to the metadata database [airflow.cli.commands
.db_command] loc=db_command.py:134 url=sqlite:////root/airflow/airflow.db
2026-10-09T16:32:08.584317Z [info     ] Context impl SQLiteImpl.       [alembic.runtime.migration] loc=mi
gration.py:205
2026-10-09T16:32:08.584714Z [info     ] Will assume non-transactional DDL. [alembic.runtime.migration] lo
c=migration.py:208
2026-10-09T16:32:10.265917Z [info     ] Context impl SQLiteImpl.       [alembic.runtime.migration] loc=mi
gration.py:205
2026-10-09T16:32:10.266224Z [info     ] Will assume non-transactional DDL. [alembic.runtime.migration] lo
c=migration.py:208
2026-10-09T16:32:10.266932Z [info     ] Creating Airflow database tables from the ORM [airflow.utils.db]
loc=db.py:749
2026-10-09T16:32:10.267179Z [info     ] Creating global lock context   [airflow.utils.db] loc=db.py:730
2026-10-09T16:32:10.267405Z [info     ] Pool status: Pool size: 5  Connections in pool: 0 Current Overflo
w: -4 Current Checked out connections: 1 [airflow.utils.db] loc=db.py:733
2026-10-09T16:32:10.267586Z [info     ] Creating metadata              [airflow.utils.db] loc=db.py:735
2026-10-09T16:32:11.179250Z [info     ] Getting alembic config         [airflow.utils.db] loc=db.py:738
2026-10-09T16:32:11.181267Z [info     ] Stamping migration head        [airflow.utils.db] loc=db.py:741
2026-10-09T16:32:11.183978Z [info     ] Context impl SQLiteImpl.       [alembic.runtime.migration] loc=mi
gration.py:205
2026-10-09T16:32:11.184302Z [info     ] Will assume non-transactional DDL. [alembic.runtime.migration] lo
c=migration.py:208
2026-10-09T16:32:11.238226Z [info     ] Running stamp_revision  -> d2f4e1b3c5a7 [alembic.runtime.migratio
n] loc=migration.py:611
2026-10-09T16:32:11.249272Z [info     ] Airflow database tables created [airflow.utils.db] loc=db.py:744
2026-10-09T16:32:11.541405Z [info     ] Database migration done!       [airflow.cli.commands.db_command]
loc=db_command.py:152
```

## Step 2: Write the Extraction Program and the DAG File

**Code:**

```
import os
print("Name: Ayushman Rajeev Verma")

extract_code = r'''
# Name: Ayushman Rajeev Verma
# extract_pipeline.py : REST API (JSON) + flat file (CSV) -> one clean report file
print("Name: Ayushman Rajeev Verma")
import json, time
import pandas as pd
import requests

API_URL = "https://jsonplaceholder.typicode.com/todos?_limit=10"
```

```
CACHED_SAMPLE = [      # cached sample of the same API response, used only when the API is unreachable
    {"userId": 1, "id": 1, "title": "delectus aut autem", "completed": False},
    {"userId": 1, "id": 2, "title": "quis ut nam facilis et officia qui", "completed": False},
    {"userId": 1, "id": 3, "title": "fugiat veniam minus", "completed": False},
    {"userId": 1, "id": 4, "title": "et porro tempora", "completed": True},
    {"userId": 1, "id": 5, "title": "laboriosam mollitia et enim quasi adipisci quia provident illum", "completed":
False},
    {"userId": 1, "id": 6, "title": "qui ullam ratione quibusdam voluptatem quia omnis", "completed": False},
    {"userId": 1, "id": 7, "title": "illo expedita consequatur quia in", "completed": False},
    {"userId": 1, "id": 8, "title": "quo adipisci enim quam ut ab", "completed": True},
    {"userId": 1, "id": 9, "title": "molestiae perspiciatis ipsa", "completed": False},
    {"userId": 1, "id": 10, "title": "illo est ratione doloremque quia maiores aut", "completed": True},
]

def fetch_json(url, retries=3, wait=1):
    """GET with retries; returns (data, source_label)."""
    for attempt in range(1, retries + 1):
        try:
            r = requests.get(url, timeout=8)
            r.raise_for_status()
            return r.json(), "live API"
        except requests.RequestException as err:
            print(f"  attempt {attempt}/{retries} failed: {type(err).__name__}")
            time.sleep(wait)
    return CACHED_SAMPLE, "cached sample (API not reachable)"

def read_categories(path):
    pd.DataFrame({
        "id": range(1, 11),
        "category": ["work", "home", "work", "study", "home", "study", "work", "health", "home", "health"],
        "due_in_days": [2, 5, 1, 7, 3, 10, 2, 4, 6, 8],
    }).to_csv(path, index=False)            # create the flat file used in this lab
    return pd.read_csv(path)

def build_report(api_rows, cat_df):
    todos = pd.json_normalize(api_rows)
    todos["status"] = todos["completed"].map({True: "Done", False: "Pending"})
    todos["title_words"] = todos["title"].str.split().str.len()
    todos["title"] = todos["title"].str.capitalize()
    report = todos.merge(cat_df, on="id", how="left", validate="one_to_one")
    return report[["id", "userId", "title", "category", "due_in_days", "title_words", "status"]]

if __name__ == "__main__":
    print("Step 1: calling the API ...")
    rows, source = fetch_json(API_URL, retries=2, wait=0)
    print("Data source:", source, "| records:", len(rows))

    print("\nStep 2: reading the CSV file")
    categories = read_categories("task_categories.csv")
    print(categories.head(3).to_string(index=False))

    print("\nStep 3: merging and transforming")
    report = build_report(rows, categories)
    pd.set_option("display.width", 130, "display.max_colwidth", 38)
    print(report.to_string(index=False))

    print("\nStep 4: summary by category")
    print(report.groupby("category").agg(tasks=("id", "count"), avg_due_days=("due_in_days", "mean"),
                                         done=("status", lambda s: (s == "Done").sum())).round(1).to_string())
    report.to_csv("todo_report.csv", index=False)
    report.to_json("todo_report.json", orient="records", indent=1)
    print("\nSaved: todo_report.csv and todo_report.json")
'''

dag_code = r'''
# Name: Ayushman Rajeev Verma
# File: ~/airflow/dags/api_csv_daily_pipeline.py
import os
from datetime import datetime, timedelta
from airflow import DAG

try:                                        # Airflow 3.x import paths
    from airflow.providers.standard.operators.bash import BashOperator
    from airflow.providers.standard.operators.empty import EmptyOperator
    from airflow.providers.standard.operators.python import PythonOperator
except ImportError:                         # Airflow 2.x import paths
    from airflow.operators.bash import BashOperator
    from airflow.operators.empty import EmptyOperator
    from airflow.operators.python import PythonOperator
```

```
SCRIPT_DIR = os.path.expanduser("~/airflow/scripts")

def quality_check():
    """Fail the task when the report is missing or has too few rows."""
    import csv
    with open(os.path.join(SCRIPT_DIR, "todo_report.csv")) as f:
        rows = list(csv.DictReader(f))
    if len(rows) < 10:
        raise ValueError(f"Only {len(rows)} rows found - expected at least 10")
    print(f"Quality check passed: {len(rows)} rows in todo_report.csv")

with DAG(
    dag_id="api_csv_daily_pipeline",
    description="Daily API + CSV extraction followed by a quality check",
    start_date=datetime(2026, 1, 1),
    schedule="0 6 * * *",                       # every day at 06:00
    catchup=False,
    tags=["practical6", "etl"],
    default_args={"owner": "ayushman", "retries": 2, "retry_delay": timedelta(minutes=2)},
) as dag:

    begin = EmptyOperator(task_id="begin")

    fetch_and_clean = BashOperator(              # runs the program from Practical 5
        task_id="fetch_and_clean",
        bash_command=f"cd {SCRIPT_DIR} && python3 extract_pipeline.py",
    )

    check_quality = PythonOperator(task_id="check_quality", python_callable=quality_check)

    notify = BashOperator(
        task_id="notify_success",
        bash_command='echo "Pipeline finished OK on $(date +%F)"',
    )

    end = EmptyOperator(task_id="end")

    begin >> fetch_and_clean >> check_quality >> notify >> end
'''

for folder in ("dags", "scripts"):
    os.makedirs(os.path.expanduser(f"~/airflow/{folder}"), exist_ok=True)
open(os.path.expanduser("~/airflow/scripts/extract_pipeline.py"), "w").write(extract_code)
open(os.path.expanduser("~/airflow/dags/api_csv_daily_pipeline.py"), "w").write(dag_code)
print("Written: ~/airflow/scripts/extract_pipeline.py")
print("Written: ~/airflow/dags/api_csv_daily_pipeline.py")
```

**Output:**

```
Name: Ayushman Rajeev Verma
Written: ~/airflow/scripts/extract_pipeline.py
Written: ~/airflow/dags/api_csv_daily_pipeline.py
```

## Step 3: Run the Extraction Program, then Validate the DAG

**Code:**

```
import subprocess
subprocess.run('cd ~/airflow/scripts && python3 extract_pipeline.py', shell=True)
```

**Output:**

```
CompletedProcess(args='cd ~/airflow/scripts && python3 extract_pipeline.py', returncode=0)
```

**Code:**

```
import os
from airflow.models import DagBag

print("Name: Ayushman Rajeev Verma")
bag = DagBag(os.path.expanduser("~/airflow/dags"))
print("Import errors   :", bag.import_errors or "None")

dag = bag.get_dag("api_csv_daily_pipeline")
print("DAG id          :", dag.dag_id)
print("Schedule (cron) :", dag.schedule)
print("Tags            :", sorted(dag.tags))
print("Default retries :", dag.default_args["retries"])
```

```
print("Tasks          :", len(dag.tasks))
print("\nDependency chain:")
for t in dag.tasks:
    nxt = ", ".join(sorted(t.downstream_task_ids)) or "(last task)"
    print(f"  {t.task_id:<16} [{type(t).__name__:<14}] --> {nxt}")
```

**Output:**

```
Name: Ayushman Rajeev Verma
2026-10-09T16:32:23.868389Z [info     ] Filling up the DagBag from /root/airflow/dags [airflow.dag_proces
sing.dagbag.DagBag] loc=dagbag.py:460
Import errors   : None
DAG id          : api_csv_daily_pipeline
Schedule (cron) : 0 6 * * *
Tags            : ['etl', 'practical6']
Default retries : 2
Tasks           : 5

Dependency chain:
  begin            [EmptyOperator ] --> fetch_and_clean
  fetch_and_clean  [BashOperator  ] --> check_quality
  check_quality    [PythonOperator] --> notify_success
  notify_success   [BashOperator  ] --> end
  end              [EmptyOperator ] --> (last task)
```

**Code:**

```
# Run the quality-check function of the DAG on its own
print("Name: Ayushman Rajeev Verma")
import importlib.util, os
spec = importlib.util.spec_from_file_location("dag_file",
os.path.expanduser("~/airflow/dags/api_csv_daily_pipeline.py"))
mod = importlib.util.module_from_spec(spec); spec.loader.exec_module(mod)
mod.quality_check()
```

**Output:**

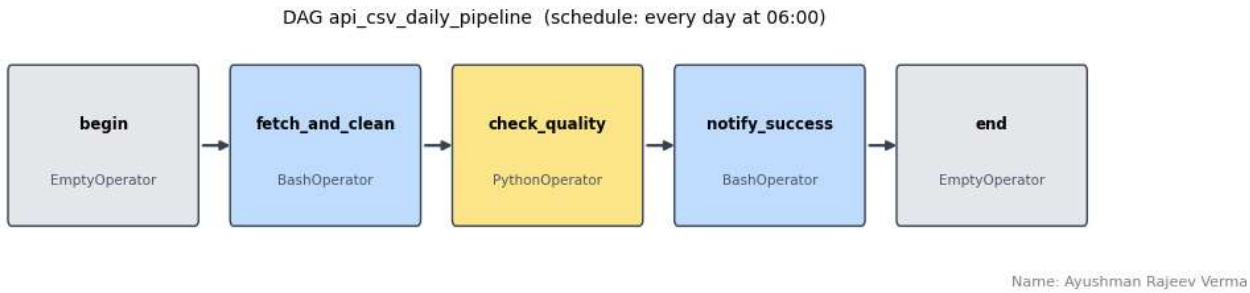
```
Name: Ayushman Rajeev Verma
Quality check passed: 10 rows in todo_report.csv
```

**Code:**

```
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch

steps = [("begin", "EmptyOperator", "#e5e7eb"), ("fetch_and_clean", "BashOperator", "#bfdbfe"),
         ("check_quality", "PythonOperator", "#fde68a"), ("notify_success", "BashOperator", "#bfdbfe"),
         ("end", "EmptyOperator", "#e5e7eb")]
fig, ax = plt.subplots(figsize=(11, 2.2))
for i, (name, kind, col) in enumerate(steps):
    x = i * 2.25
    ax.add_patch(FancyBboxPatch((x, 0.25), 1.85, 1.0, boxstyle="round,pad=0.04", fc=col, ec="#374151"))
    ax.text(x + 0.925, 0.9, name, ha="center", va="center", fontsize=8.5, fontweight="bold")
    ax.text(x + 0.925, 0.52, kind, ha="center", va="center", fontsize=7.5, color="#4b5563")
    if i < len(steps) - 1:
        ax.annotate("", xy=(x + 2.25, 0.75), xytext=(x + 1.9, 0.75), arrowprops=dict(arrowstyle="-|>", lw=1.6,
color="#374151"))
ax.set_xlim(-0.2, 11.2); ax.set_ylim(0, 1.5); ax.axis("off")
ax.set_title("DAG api_csv_daily_pipeline  (schedule: every day at 06:00)", fontsize=10)
plt.gcf().text(0.99, 0.005, "Name: Ayushman Rajeev Verma", ha="right", va="bottom", fontsize=8, color="gray")
plt.savefig("p6_dag.png", dpi=120, bbox_inches="tight")
```

**Output:**



### Step 4: Run the DAG

Run the following command in a terminal, open http://localhost:8080, unpause the DAG api_csv_daily_pipeline and trigger it. All five tasks turn green when the run succeeds.

**Code:**

```
airflow standalone
```

# Practical No. 7: ETL Pipelines: CSV and JSON Sources, Invalid Records, Validation and Incremental Loading

**Aim:** To build ETL pipelines for an HR payroll data set that extract data from one CSV file, from several CSV files and from a JSON document, clean and transform the data, find and remove invalid records, validate batches before loading, load them into a relational database and perform incremental loads using a watermark.

## Step 1: Create the Source Files and the Target Database

**Code:**

```
# Name: Ayushman Rajeev Verma
print("Name: Ayushman Rajeev Verma")
import os, glob, json, sqlite3
import numpy as np
import pandas as pd

HEAD = "emp_id,name,dept,monthly_salary,join_date,updated_at\n"
files = {
 "payroll_it.csv": [
    "E101,  aarav mehta ,IT,65000,2021-06-15,2026-01-05 10:00:00",
    "E102,ISHA NAIR,IT,72000,2019-03-01,2026-01-05 10:00:00",
    "E103,kabir singh,IT,-52000,2022-09-12,2026-01-06 09:30:00",          # negative salary
    "E101,  aarav mehta ,IT,65000,2021-06-15,2026-01-05 10:00:00"],       # duplicate row
 "payroll_hr.csv": [
    "E201,Meera Joshi,HR,48000,2020-11-23,2026-01-05 11:00:00",
    "E202,nikhil rao,HR,,2023-01-09,2026-01-05 11:00:00",                  # missing salary
    "E203,Pooja Desai,HR,51000,31-02-2022,2026-01-07 08:15:00"],          # invalid date
 "payroll_sales.csv": [
    "E301,Sameer Khan,Sales,41000,2022-07-18,2026-01-05 12:00:00",
    "E302,Tara Iyer,Sales,39000,2024-02-05,2026-01-08 14:20:00",
    "E303,Uday Patil,Sales,44500,2021-10-30,2026-01-08 14:45:00"],
}
for fname, lines in files.items():
    with open(fname, "w") as f:
        f.write(HEAD + "\n".join(lines) + "\n")

attendance = {"month": "2026-01", "working_days": 22, "records": [
    {"emp_id": "E101", "days_present": 21, "leaves": {"casual": 1, "sick": 0}},
    {"emp_id": "E102", "days_present": 22, "leaves": {"casual": 0, "sick": 0}},
    {"emp_id": "E201", "days_present": 18, "leaves": {"casual": 2, "sick": 2}},
    {"emp_id": "E301", "days_present": 20, "leaves": {"casual": 1, "sick": 1}},
    {"emp_id": "E999", "days_present": 22, "leaves": {"casual": 0, "sick": 0}},       # unknown employee
]}
json.dump(attendance, open("attendance.json", "w"), indent=2)

if os.path.exists("payroll_dw.db"):
    os.remove("payroll_dw.db")
db = sqlite3.connect("payroll_dw.db")
print("Created 3 department CSV files, attendance.json and the target database payroll_dw.db")
```

**Output:**

```
Name: Ayushman Rajeev Verma
Created 3 department CSV files, attendance.json and the target database payroll_dw.db
```

## Step 2: ETL for a Single CSV File

Three functions are reused in all later tasks: standardise() fixes text, dates and duplicates; enrich() adds annual salary, tax slab and tenure; to_db_types() prepares dates for SQLite.

**Code:**

```
# ---- reusable transformation functions
print("Name: Ayushman Rajeev Verma")
def standardise(df):
    out = df.copy()
    out["name"] = out["name"].str.strip().str.title()
    out["dept"] = out["dept"].str.strip().str.upper()
    out["join_date"] = pd.to_datetime(out["join_date"], format="%Y-%m-%d", errors="coerce")
    out["updated_at"] = pd.to_datetime(out["updated_at"])
    return out.drop_duplicates()
```

```
def enrich(df):
    out = df.copy()
    out["annual_salary"] = out["monthly_salary"] * 12
    out["tax_pct"] = np.select([out["monthly_salary"] <= 40000, out["monthly_salary"] <= 70000], [5, 10],
default=20)
    out["tenure_years"] = ((pd.Timestamp("2026-04-01") - out["join_date"]).dt.days / 365.25).round(1)
    return out

def to_db_types(df):                        # SQLite stores dates as text
    out = df.copy()
    out["join_date"] = out["join_date"].dt.strftime("%Y-%m-%d")
    out["updated_at"] = out["updated_at"].dt.strftime("%Y-%m-%d %H:%M:%S")
    return out

# ---- E X T R A C T
raw_it = pd.read_csv("payroll_it.csv")
print("Extracted:", len(raw_it), "rows\n", raw_it.to_string(index=False), "\n")

# ---- T R A N S F O R M
it = enrich(standardise(raw_it))
print("Transformed:", len(it), "rows (duplicate removed)\n", it[["emp_id", "name", "dept", "monthly_salary",
"tax_pct", "tenure_years"]].to_string(index=False), "\n")

# ---- L O A D
to_db_types(it).to_sql("payroll_it", db, if_exists="replace", index=False)
print("Loaded into table payroll_it ->", db.execute("SELECT COUNT(*) FROM payroll_it").fetchone()[0], "rows")
```

**Output:**

```
Name: Ayushman Rajeev Verma
Extracted: 4 rows
 emp_id           name dept  monthly_salary  join_date          updated_at
  E101   aarav mehta    IT           65000 2021-06-15 2026-01-05 10:00:00
  E102      ISHA NAIR   IT           72000 2019-03-01 2026-01-05 10:00:00
  E103    kabir singh   IT          -52000 2022-09-12 2026-01-06 09:30:00
  E101   aarav mehta    IT           65000 2021-06-15 2026-01-05 10:00:00

Transformed: 3 rows (duplicate removed)
 emp_id        name dept  monthly_salary  tax_pct  tenure_years
  E101 Aarav Mehta   IT           65000       10           4.8
  E102   Isha Nair   IT           72000       20           7.1
  E103 Kabir Singh   IT          -52000        5           3.6

Loaded into table payroll_it -> 3 rows
```

## Step 3: Extract Multiple CSV Files and Combine Them

**Code:**

```
print("Name: Ayushman Rajeev Verma")
parts = {}
for path in sorted(glob.glob("payroll_*.csv")):
    parts[os.path.basename(path)] = pd.read_csv(path)
    print(f"{path:<18} rows: {len(parts[path]):>2}")

combined = pd.concat(parts, names=["source_file", "row"]).reset_index(level="source_file")    # keeps the file name
print("\nRows combined:", len(combined))
combined = enrich(standardise(combined))
print("Rows after standardise + de-duplicate:", len(combined))

to_db_types(combined).to_sql("payroll_all", db, if_exists="replace", index=False)
print("\nRows per department in table payroll_all:")
print(pd.read_sql("SELECT dept, COUNT(*) AS employees, ROUND(AVG(monthly_salary)) AS avg_salary FROM payroll_all
GROUP BY dept", db).to_string(index=False))
```

**Output:**

```
Name: Ayushman Rajeev Verma
payroll_hr.csv     rows:  3
payroll_it.csv     rows:  4
payroll_sales.csv  rows:  3

Rows combined: 10
Rows after standardise + de-duplicate: 9

Rows per department in table payroll_all:
 dept  employees  avg_salary
   HR          3     49500.0
   IT          3     28333.0
SALES          3     41500.0
```

## Step 4: Extract and Flatten a JSON Document

**Code:**

```
print("Name: Ayushman Rajeev Verma")
doc = json.load(open("attendance.json"))
att = pd.json_normalize(doc, record_path="records", meta=["month", "working_days"], sep="_")   # flatten nested
leaves
att["attendance_pct"] = (att["days_present"] / att["working_days"] * 100).round(1)
att["total_leaves"] = att["leaves_casual"] + att["leaves_sick"]
att = att[["emp_id", "month", "days_present", "total_leaves", "attendance_pct"]]
print(att.to_string(index=False))

att.to_sql("attendance", db, if_exists="replace", index=False)
print("\nJoin check - attendance of known employees only:")
print(pd.read_sql("""SELECT a.emp_id, p.name, p.dept, a.attendance_pct FROM attendance a
                     JOIN payroll_all p ON p.emp_id = a.emp_id ORDER BY a.emp_id""", db).to_string(index=False))
```

**Output:**

```
Name: Ayushman Rajeev Verma
emp_id   month  days_present  total_leaves attendance_pct
  E101 2026-01            21             1      95.454545
  E102 2026-01            22             0          100.0
  E201 2026-01            18             4      81.818182
  E301 2026-01            20             2      90.909091
  E999 2026-01            22             0          100.0

Join check - attendance of known employees only:
emp_id        name  dept  attendance_pct
  E101 Aarav Mehta    IT       95.454545
  E102   Isha Nair    IT      100.000000
  E201 Meera Joshi    HR       81.818182
  E301 Sameer Khan SALES       90.909091
```

## Step 5: Identify and Remove Invalid Records (Rule Based)

Every rule is a message plus a function. A row that breaks a rule goes to the payroll_rejects table together with the reasons.

**Code:**

```
# Each rule is (message, function that is True for a valid row)
print("Name: Ayushman Rajeev Verma")
RULES = {
    "monthly_salary must be a positive number": lambda r: pd.notna(r["monthly_salary"]) and r["monthly_salary"] > 0,
    "join_date must be a valid date":           lambda r: pd.notna(r["join_date"]),
    "name must not be empty":                   lambda r: bool(str(r["name"]).strip()),
}

def split_valid_invalid(df):
    reasons = df.apply(lambda r: "; ".join(msg for msg, ok in RULES.items() if not ok(r)), axis=1)
    return df[reasons == ""], df[reasons != ""].assign(reason=reasons[reasons != ""])

valid, invalid = split_valid_invalid(combined)
print(f"Total: {len(combined)} | valid: {len(valid)} | invalid: {len(invalid)}\n")
print("Rejected records:")
print(invalid[["emp_id", "name", "reason"]].to_string(index=False))
```

```
to_db_types(invalid).to_sql("payroll_rejects", db, if_exists="replace", index=False)
```

**Output:**

```
Name: Ayushman Rajeev Verma
Total: 9 | valid: 6 | invalid: 3

Rejected records:
emp_id        name                                    reason
  E202  Nikhil Rao monthly_salary must be a positive number
  E203 Pooja Desai           join_date must be a valid date
  E103 Kabir Singh monthly_salary must be a positive number
```

## Step 6: Validate Data Before Loading to the Target Database

validate_batch() raises a ValidationError when a batch is unsafe to load; the load function catches it, reports the reason and loads nothing. The target table also has a primary key and a CHECK constraint.

**Code:**

```
print("Name: Ayushman Rajeev Verma")
class ValidationError(Exception):
    pass

REQUIRED = ["emp_id", "name", "dept", "monthly_salary", "join_date", "updated_at"]

def validate_batch(df):
    """Raises ValidationError when the batch must NOT be loaded."""
    missing = [c for c in REQUIRED if c not in df.columns]
    if missing:
        raise ValidationError(f"missing columns {missing}")
    problems = []
    if df["emp_id"].duplicated().any():          problems.append("duplicate emp_id")
    if df["monthly_salary"].isna().any() or (df["monthly_salary"] <= 0).any():
        problems.append("invalid salary")
    if problems:
        raise ValidationError("; ".join(problems))

db.executescript("""
DROP TABLE IF EXISTS payroll_clean;
CREATE TABLE payroll_clean (emp_id TEXT PRIMARY KEY, name TEXT NOT NULL, dept TEXT, monthly_salary REAL CHECK
(monthly_salary > 0),
    join_date TEXT, updated_at TEXT, annual_salary REAL, tax_pct INTEGER, tenure_years REAL);
""")
COLS = ["emp_id", "name", "dept", "monthly_salary", "join_date", "updated_at", "annual_salary", "tax_pct",
"tenure_years"]

def guarded_load(batch, label):
    try:
        validate_batch(batch)
    except ValidationError as err:
        print(f"[{label}] REJECTED  -> {err}")
        return
    to_db_types(batch)[COLS].to_sql("payroll_clean", db, if_exists="append", index=False)
    print(f"[{label}] ACCEPTED  -> {len(batch)} rows loaded")

guarded_load(valid.drop(columns=["monthly_salary"]), "batch 1: salary column missing")
guarded_load(pd.concat([valid, valid.head(1)]), "batch 2: repeated employee")
guarded_load(invalid.drop(columns="reason"), "batch 3: bad salaries")
guarded_load(valid, "batch 4: clean data")
print("\nTarget table payroll_clean:")
print(pd.read_sql("SELECT emp_id, name, dept, monthly_salary, tax_pct, updated_at FROM payroll_clean",
db).to_string(index=False))
```

**Output:**

```
Name: Ayushman Rajeev Verma
[batch 1: salary column missing] REJECTED  -> missing columns ['monthly_salary']
[batch 2: repeated employee] REJECTED  -> duplicate emp_id
[batch 3: bad salaries] REJECTED  -> invalid salary
[batch 4: clean data] ACCEPTED  -> 6 rows loaded

Target table payroll_clean:
emp_id        name  dept  monthly_salary  tax_pct          updated_at
  E201 Meera Joshi    HR         48000.0       10 2026-01-05 11:00:00
  E101 Aarav Mehta    IT         65000.0       10 2026-01-05 10:00:00
  E102   Isha Nair    IT         72000.0       20 2026-01-05 10:00:00
  E301 Sameer Khan SALES         41000.0       10 2026-01-05 12:00:00
  E302   Tara Iyer SALES         39000.0        5 2026-01-08 14:20:00
  E303  Uday Patil SALES         44500.0       10 2026-01-08 14:45:00
```

## Step 7: Incremental Loading with a Watermark

The watermark table stores the newest updated_at that was loaded. A new run only processes rows that are newer, updates changed employees and inserts new ones.

**Code:**

```
# Incremental load using a WATERMARK (the newest updated_at that was already loaded)
print("Name: Ayushman Rajeev Verma")
db.execute("CREATE TABLE IF NOT EXISTS etl_watermark (table_name TEXT PRIMARY KEY, last_loaded TEXT)")
db.execute("INSERT OR REPLACE INTO etl_watermark SELECT 'payroll_clean', MAX(updated_at) FROM payroll_clean")
db.commit()

with open("payroll_update.csv", "w") as f:
    f.write(HEAD + "\n".join([
        "E101,Aarav Mehta,IT,68000,2021-06-15,2026-01-10 09:00:00",     # salary revised -> changed
        "E302,Tara Iyer,Sales,39000,2024-02-05,2026-01-08 14:20:00",    # unchanged (older than watermark)
        "E304,Vikram Sethi,Sales,46000,2025-12-01,2026-01-11 10:00:00", # new employee
        "E201,Meera Joshi,HR,50000,2020-11-23,2026-01-12 16:30:00"]) + "\n")  # salary revised

def incremental_load(path):
    mark = db.execute("SELECT last_loaded FROM etl_watermark WHERE table_name = 'payroll_clean'").fetchone()[0]
    incoming = enrich(standardise(pd.read_csv(path)))
    fresh = incoming[incoming["updated_at"] > pd.Timestamp(mark)]            # only rows newer than the watermark
    print(f"  watermark = {mark} | incoming = {len(incoming)} | newer = {len(fresh)} | skipped = {len(incoming) -
len(fresh)}")
    if fresh.empty:
        return
    validate_batch(fresh)
    db.executemany("""INSERT INTO payroll_clean VALUES (?,?,?,?,?,?,?,?,?) ON CONFLICT(emp_id) DO UPDATE SET
                      monthly_salary = excluded.monthly_salary, updated_at = excluded.updated_at,
                      annual_salary = excluded.annual_salary, tax_pct = excluded.tax_pct""",
                   to_db_types(fresh)[COLS].astype(object).values.tolist())
    db.execute("UPDATE etl_watermark SET last_loaded = (SELECT MAX(updated_at) FROM payroll_clean) WHERE table_name
= 'payroll_clean'")
    db.commit()

print("Rows before:", db.execute("SELECT COUNT(*) FROM payroll_clean").fetchone()[0])
print("Run 1 - new file arrives:")
incremental_load("payroll_update.csv")
print("Run 2 - same file again:")
incremental_load("payroll_update.csv")
print("\nRows after:", db.execute("SELECT COUNT(*) FROM payroll_clean").fetchone()[0])
print(pd.read_sql("SELECT emp_id, name, monthly_salary, tax_pct, updated_at FROM payroll_clean ORDER BY emp_id",
db).to_string(index=False))
```

**Output:**

```
Name: Ayushman Rajeev Verma
Rows before: 6
Run 1 - new file arrives:
  watermark = 2026-01-08 14:45:00 | incoming = 4 | newer = 3 | skipped = 1
Run 2 - same file again:
  watermark = 2026-01-12 16:30:00 | incoming = 4 | newer = 0 | skipped = 4

Rows after: 7
emp_id         name  monthly_salary  tax_pct          updated_at
  E101  Aarav Mehta         68000.0       10 2026-01-10 09:00:00
  E102    Isha Nair         72000.0       20 2026-01-05 10:00:00
  E201  Meera Joshi         50000.0       10 2026-01-12 16:30:00
  E301  Sameer Khan         41000.0       10 2026-01-05 12:00:00
  E302    Tara Iyer         39000.0        5 2026-01-08 14:20:00
  E303   Uday Patil         44500.0       10 2026-01-08 14:45:00
  E304 Vikram Sethi         46000.0       10 2026-01-11 10:00:00
```

# Practical No. 8: Distributed Data Processing with PySpark DataFrames

**Aim:** To use PySpark to read a CSV data set with an explicit schema, perform filtering, grouping and aggregation, remove duplicate records, join two data sets and calculate the average salary of every department (with the DataFrame API and with Spark SQL).

Run this cell once if PySpark is not installed (Google Colab already has Java).

**Code:**

```
%pip install -q pyspark
```

## Step 1: Start a Spark Session and Create the Input Files

**Code:**

```
# Name: Ayushman Rajeev Verma
print("Name: Ayushman Rajeev Verma")
from pyspark.sql import SparkSession, functions as F, Window
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType

spark = (SparkSession.builder.appName("Practical8_Employees").master("local[2]")
         .config("spark.ui.enabled", "false").config("spark.sql.shuffle.partitions", "4").getOrCreate())
spark.sparkContext.setLogLevel("ERROR")
print("Spark version:", spark.version)

emp_rows = """101,Aarav Mehta,IT,65000,Pune,2021
102,Isha Nair,HR,48000,Mumbai,2019
103,Kabir Singh,IT,52000,Pune,2022
104,Meera Joshi,Finance,58000,Nashik,2020
105,Nikhil Rao,Finance,72000,Mumbai,2018
106,Pooja Desai,Sales,41000,Pune,2023
107,Sameer Khan,Sales,39000,Nashik,2024
102,Isha Nair,HR,48000,Mumbai,2019
108,Tara Iyer,IT,91000,Mumbai,2017
109,Uday Patil,Legal,60000,Pune,2022
106,Pooja Desai,Sales,41000,Pune,2023
110,Vikram Sethi,Sales,47000,Mumbai,2021"""
with open("employees.csv", "w") as f:
    f.write("emp_id,name,dept,salary,city,join_year\n" + emp_rows + "\n")
with open("departments.csv", "w") as f:
    f.write("dept,head,budget\nIT,Rohan Kulkarni,5000000\nHR,Anita Rao,2000000\nFinance,Sanjay
Gupta,3500000\nSales,Neha Kapoor,4000000\n")
```

**Output:**

```
Name: Ayushman Rajeev Verma
Spark version: 4.0.4
```

## Step 2: Read a CSV Data Set with an Explicit Schema

**Code:**

```
# Read the CSV file with an explicit schema (no guessing) and display it
print("Name: Ayushman Rajeev Verma")
schema = StructType([
    StructField("emp_id", IntegerType(), False), StructField("name", StringType(), True),
    StructField("dept", StringType(), True),     StructField("salary", DoubleType(), True),
    StructField("city", StringType(), True),     StructField("join_year", IntegerType(), True)])

emp = spark.read.csv("employees.csv", header=True, schema=schema)
dept = spark.read.csv("departments.csv", header=True, inferSchema=True)

emp.printSchema()
print("Employees:", emp.count(), "rows | Departments:", dept.count(), "rows")
emp.show()
```

**Output:**

```
Name: Ayushman Rajeev Verma
root
 |-- emp_id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- dept: string (nullable = true)
 |-- salary: double (nullable = true)
 |-- city: string (nullable = true)
 |-- join_year: integer (nullable = true)

Employees: 12 rows | Departments: 4 rows
+------+------------+-------+-------+------+---------+
|emp_id|        name|   dept| salary|  city|join_year|
+------+------------+-------+-------+------+---------+
|   101| Aarav Mehta|     IT|65000.0|  Pune|     2021|
|   102|   Isha Nair|     HR|48000.0|Mumbai|     2019|
|   103| Kabir Singh|     IT|52000.0|  Pune|     2022|
|   104| Meera Joshi|Finance|58000.0|Nashik|     2020|
|   105|  Nikhil Rao|Finance|72000.0|Mumbai|     2018|
|   106| Pooja Desai|  Sales|41000.0|  Pune|     2023|
|   107| Sameer Khan|  Sales|39000.0|Nashik|     2024|
|   102|   Isha Nair|     HR|48000.0|Mumbai|     2019|
|   108|   Tara Iyer|     IT|91000.0|Mumbai|     2017|
|   109|  Uday Patil|  Legal|60000.0|  Pune|     2022|
|   106| Pooja Desai|  Sales|41000.0|  Pune|     2023|
|   110|Vikram Sethi|  Sales|47000.0|Mumbai|     2021|
+------+------------+-------+-------+------+---------+
```

## Step 3: Filtering, Grouping and Aggregation

**Code:**

```
# FILTERING - column expressions and SQL-style conditions
print("Name: Ayushman Rajeev Verma")
print("Employees in Pune earning more than 50000:")
emp.filter((F.col("city") == "Pune") & (F.col("salary") > 50000)).select("name", "dept", "salary", "city").show()

print("Salary between 40000 and 60000, joined after 2020:")
emp.where("salary BETWEEN 40000 AND 60000 AND join_year > 2020").select("name", "salary", "join_year").show()

# GROUPING and AGGREGATION
print("Department summary:")
(emp.groupBy("dept")
    .agg(F.count("*").alias("headcount"), F.round(F.avg("salary"), 1).alias("avg_salary"),
         F.max("salary").alias("highest"), F.sum("salary").alias("payroll"))
    .orderBy(F.desc("payroll")).show())
```

**Output:**

```
Name: Ayushman Rajeev Verma
Employees in Pune earning more than 50000:
+-----------+-----+-------+----+
|       name| dept| salary|city|
+-----------+-----+-------+----+
|Aarav Mehta|   IT|65000.0|Pune|
|Kabir Singh|   IT|52000.0|Pune|
| Uday Patil|Legal|60000.0|Pune|
+-----------+-----+-------+----+

Salary between 40000 and 60000, joined after 2020:
+------------+-------+---------+
|        name| salary|join_year|
+------------+-------+---------+
| Kabir Singh|52000.0|     2022|
| Pooja Desai|41000.0|     2023|
|  Uday Patil|60000.0|     2022|
| Pooja Desai|41000.0|     2023|
|Vikram Sethi|47000.0|     2021|
+------------+-------+---------+

Department summary:
+-------+---------+----------+-------+--------+
|   dept|headcount|avg_salary|highest| payroll|
+-------+---------+----------+-------+--------+
|     IT|        3|   69333.3|91000.0|208000.0|
|  Sales|        4|   42000.0|47000.0|168000.0|
|Finance|        2|   65000.0|72000.0|130000.0|
|     HR|        2|   48000.0|48000.0| 96000.0|
|  Legal|        1|   60000.0|60000.0| 60000.0|
+-------+---------+----------+-------+--------+
```

## Step 4: Find and Remove Duplicate Records

**Code:**

```
# DUPLICATE RECORDS - first find them, then remove them
print("Name: Ayushman Rajeev Verma")
dups = emp.groupBy(emp.columns).count().filter("count > 1")
print("Duplicate records found:")
dups.show()

emp_clean = emp.dropDuplicates()
print("Rows before:", emp.count(), "| after dropDuplicates():", emp_clean.count())
print("Distinct emp_id values:", emp_clean.select("emp_id").distinct().count())
```

**Output:**

```
Name: Ayushman Rajeev Verma
Duplicate records found:
+------+-----------+-----+-------+------+---------+-----+
|emp_id|       name| dept| salary|  city|join_year|count|
+------+-----------+-----+-------+------+---------+-----+
|   102|  Isha Nair|   HR|48000.0|Mumbai|     2019|    2|
|   106|Pooja Desai|Sales|41000.0|  Pune|     2023|    2|
+------+-----------+-----+-------+------+---------+-----+

Rows before: 12 | after dropDuplicates(): 10
Distinct emp_id values: 10
```

## Step 5: Join Two Data Sets

**Code:**

```
# JOINS
print("Name: Ayushman Rajeev Verma")
inner = emp_clean.join(dept, on="dept", how="inner")
print("Inner join - employees with their department head:")
inner.select("emp_id", "name", "dept", "head", "salary").orderBy("emp_id").show()

print("Left anti join - employees whose department is missing in departments.csv:")
emp_clean.join(dept, on="dept", how="left_anti").select("emp_id", "name", "dept").show()
```

```
print("Department budget per employee:")
(inner.groupBy("dept", "budget").agg(F.count("*").alias("staff"))
      .withColumn("budget_per_employee", F.round(F.col("budget") / F.col("staff"))).orderBy("dept").show())
```

**Output:**

```
Name: Ayushman Rajeev Verma
Inner join - employees with their department head:
+------+------------+-------+--------------+-------+
|emp_id|        name|   dept|          head| salary|
+------+------------+-------+--------------+-------+
|   101| Aarav Mehta|     IT|Rohan Kulkarni|65000.0|
|   102|   Isha Nair|     HR|     Anita Rao|48000.0|
|   103| Kabir Singh|     IT|Rohan Kulkarni|52000.0|
|   104| Meera Joshi|Finance|  Sanjay Gupta|58000.0|
|   105|  Nikhil Rao|Finance|  Sanjay Gupta|72000.0|
|   106| Pooja Desai|  Sales|   Neha Kapoor|41000.0|
|   107| Sameer Khan|  Sales|   Neha Kapoor|39000.0|
|   108|   Tara Iyer|     IT|Rohan Kulkarni|91000.0|
|   110|Vikram Sethi|  Sales|   Neha Kapoor|47000.0|
+------+------------+-------+--------------+-------+

Left anti join - employees whose department is missing in departments.csv:
+------+----------+-----+
|emp_id|      name| dept|
+------+----------+-----+
|   109|Uday Patil|Legal|
+------+----------+-----+

Department budget per employee:
+-------+-------+-----+------------------+
|   dept| budget|staff|budget_per_employee|
+-------+-------+-----+------------------+
|Finance|3500000|    2|         1750000.0|
|     HR|2000000|    1|         2000000.0|
|     IT|5000000|    3|         1666667.0|
|  Sales|4000000|    3|         1333333.0|
+-------+-------+-----+------------------+
```

## Step 6: Average Salary by Department (DataFrame API and Spark SQL)

**Code:**

```
# AVERAGE SALARY BY DEPARTMENT - DataFrame API
print("Name: Ayushman Rajeev Verma")
avg_df = emp_clean.groupBy("dept").agg(F.round(F.avg("salary"),
2).alias("average_salary")).orderBy(F.desc("average_salary"))
avg_df.show()

# the same result with Spark SQL
emp_clean.createOrReplaceTempView("employees")
spark.sql("SELECT dept, ROUND(AVG(salary), 2) AS average_salary FROM employees GROUP BY dept ORDER BY average_salary
DESC").show()

# extra: highest paid employee in every department (window function)
w = Window.partitionBy("dept").orderBy(F.desc("salary"))
emp_clean.withColumn("rank_in_dept", F.rank().over(w)).filter("rank_in_dept = 1").select("dept", "name",
"salary").orderBy("dept").show()
spark.stop()
```

**Output:**

```
Name: Ayushman Rajeev Verma
+-------+--------------+
|   dept|average_salary|
+-------+--------------+
|     IT|      69333.33|
|Finance|       65000.0|
|  Legal|       60000.0|
|     HR|       48000.0|
|  Sales|      42333.33|
+-------+--------------+

+-------+--------------+
|   dept|average_salary|
+-------+--------------+
|     IT|      69333.33|
|Finance|       65000.0|
|  Legal|       60000.0|
|     HR|       48000.0|
|  Sales|      42333.33|
+-------+--------------+

+-------+------------+-------+
|   dept|        name| salary|
+-------+------------+-------+
|Finance|  Nikhil Rao|72000.0|
|     HR|   Isha Nair|48000.0|
|     IT|   Tara Iyer|91000.0|
|  Legal|  Uday Patil|60000.0|
|  Sales|Vikram Sethi|47000.0|
+-------+------------+-------+
```

# Practical No. 9: End-to-End Data Pipelines: E-Commerce ETL with Historical and Newly Arriving Data

**Aim:** To build an end-to-end pipeline from a CSV file through Pandas cleaning and a SQL database to a report; to build an ETL pipeline for an e-commerce data set with customers, products, orders and payments; and to design a pipeline that processes historical data and newly arriving files without loading anything twice.

## Step 1: Pipeline 1 - CSV to Pandas to Cleaning to SQL Database to Report

**Code:**

```
# Name: Ayushman Rajeev Verma
# PIPELINE 1 : movie-theatre bookings  CSV -> Pandas (method chaining) -> SQL database -> report
print("Name: Ayushman Rajeev Verma")
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt

with open("bookings_raw.csv", "w") as f:
    f.write("booking_id,show_date,movie,seats,price_per_seat,screen\n")
    f.write("\n".join([
        "B1,2026-03-02,  inception ,3,250,Screen 1", "B2,2026-03-02,INTERSTELLAR,2,300,Screen 2",
        "B3,2026-03-03,Inception,4,250,Screen 1",    "B3,2026-03-03,Inception,4,250,Screen 1",
        "B4,2026-03-04,Dune,-2,280,Screen 3",        "B5,2026-03-05,Interstellar,5,,Screen 2",
        "B6,2026-03-06,dune,6,280,Screen 3",         "B7,2026-03-07,Inception,2,250,Screen 1",
        "B8,31-13-2026,Dune,2,280,Screen 3",         "B9,2026-03-08,Interstellar,3,300,Screen 2"]) + "\n")

# small, named cleaning steps that are chained with .pipe()
def tidy_text(d):    return d.assign(movie=d["movie"].str.strip().str.title(), screen=d["screen"].str.strip())
def parse_dates(d):  return d.assign(show_date=pd.to_datetime(d["show_date"], format="%Y-%m-%d", errors="coerce"))
def drop_invalid(d): return d.dropna(subset=["show_date", "price_per_seat"]).query("seats > 0")
def add_revenue(d):  return d.assign(revenue=d["seats"] * d["price_per_seat"], weekday=d["show_date"].dt.day_name())

raw = pd.read_csv("bookings_raw.csv")
clean = (raw.drop_duplicates().pipe(tidy_text).pipe(parse_dates).pipe(drop_invalid).pipe(add_revenue))
print(f"Rows: raw = {len(raw)}, clean = {len(clean)}  ->  {len(raw) - len(clean)} rows removed")
print(clean[["booking_id", "show_date", "movie", "seats", "revenue", "weekday"]].to_string(index=False))

con1 = sqlite3.connect("theatre.db")
clean.assign(show_date=clean["show_date"].dt.strftime("%Y-%m-%d")).to_sql("bookings", con1, if_exists="replace",
index=False)

report = pd.read_sql("""SELECT movie, COUNT(*) AS bookings, SUM(seats) AS seats_sold, SUM(revenue) AS revenue,
                            ROUND(AVG(price_per_seat), 1) AS avg_price
                        FROM bookings GROUP BY movie ORDER BY revenue DESC""", con1)
print("\nREPORT - revenue by movie")
print(report.to_string(index=False))
by_day = pd.read_sql("SELECT weekday, SUM(revenue) AS revenue FROM bookings GROUP BY weekday ORDER BY revenue DESC",
con1)
print("\nREPORT - best selling weekday:", by_day.iloc[0, 0], "with Rs.", int(by_day.iloc[0, 1]))
report.to_csv("theatre_report.csv", index=False)

ax = report.plot(kind="barh", x="movie", y="revenue", color="#8e44ad", legend=False, figsize=(6, 2.8))
ax.set_title("Revenue by movie (Rs.)"); ax.invert_yaxis()
plt.gcf().text(0.99, 0.005, "Name: Ayushman Rajeev Verma", ha="right", va="bottom", fontsize=8, color="gray")
plt.savefig("p9_theatre.png", dpi=110, bbox_inches="tight")
plt.show()
```

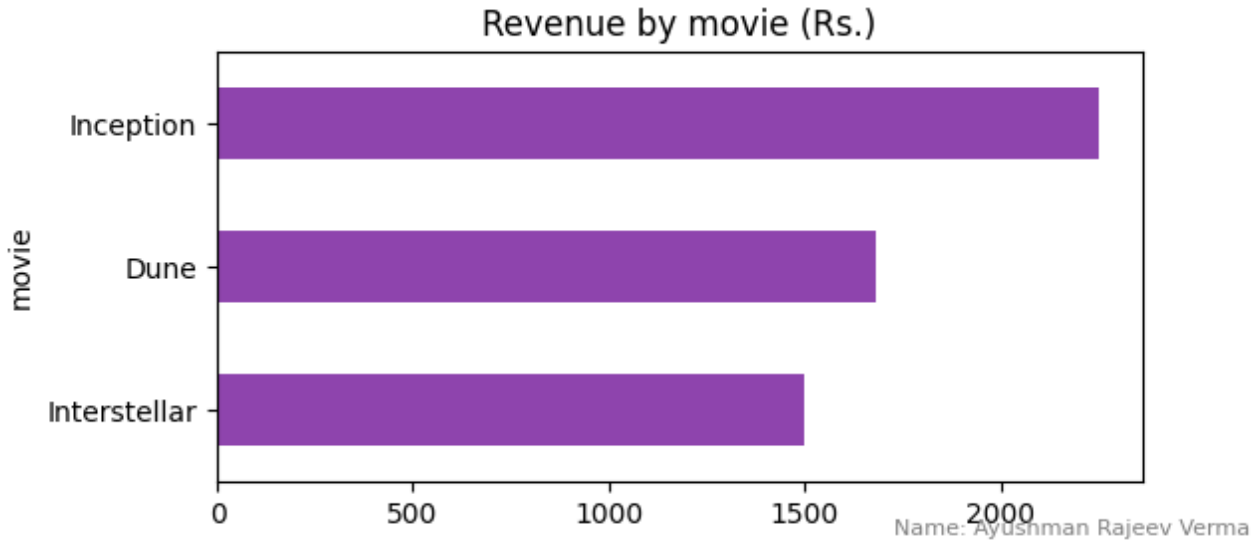
**Output:**

```
Name: Ayushman Rajeev Verma
Rows: raw = 10, clean = 6  ->  4 rows removed
booking_id  show_date        movie  seats  revenue  weekday
        B1 2026-03-02    Inception      3    750.0   Monday
        B2 2026-03-02 Interstellar      2    600.0   Monday
        B3 2026-03-03    Inception      4   1000.0  Tuesday
        B6 2026-03-06         Dune      6   1680.0   Friday
        B7 2026-03-07    Inception      2    500.0 Saturday
        B9 2026-03-08 Interstellar      3    900.0   Sunday

REPORT - revenue by movie
       movie  bookings  seats_sold  revenue  avg_price
   Inception         3           9   2250.0      250.0
        Dune         1           6   1680.0      280.0
Interstellar         2           5   1500.0      300.0

REPORT - best selling weekday: Friday with Rs. 1680
```

**Output:**




## Step 2: Pipeline 2 - Create the E-Commerce Source Data (Master Data and Inbox Files)

Customers and products are master data. Orders (with their payment details) arrive as files in an inbox folder: one historical monthly file and daily files that come later.

**Code:**

```
# PIPELINE 2 : GadgetHub e-commerce ETL  (customers, products, orders, payments)
print("Name: Ayushman Rajeev Verma")
import os, glob

os.makedirs("inbox", exist_ok=True)
customers_df = pd.DataFrame({
    "customer_id": ["C01", "C02", "C03", "C04", "C05"],
    "name": ["aman gupta", "Bhavna Rao", "chirag shah", "Divya Menon", "Esha Kapoor"],
    "email": ["aman@mail.com", "bhavna@mail.com", "CHIRAG@MAIL.COM", "divya@mail", "esha@mail.com"],
    "city": ["Mumbai", "Pune", "Nashik", "Delhi", "Pune"]})
products_df = pd.DataFrame({
    "sku": ["G1", "G2", "G3", "G4", "G5"],
    "product_name": ["Wireless Earbuds", "Smart Watch", "Power Bank 20000mAh", "Bluetooth Speaker", "USB-C Hub"],
    "category": ["Audio", "Wearables", "Accessories", "Audio", "Accessories"],
    "price": [2499, 6999, 1799, 3299, 1299]})

BATCH_HEADER = "order_id,customer_id,sku,qty,order_date,status,pay_amount,pay_mode,pay_status\n"
def write_batch(name, rows):
    with open(f"inbox/{name}", "w") as f:
        f.write(BATCH_HEADER + "\n".join(rows) + "\n")

write_batch("orders_2026-01.csv", [                     # HISTORICAL data (a whole month)
    "5001,C01,G1,1,2026-01-12,delivered,2499,UPI,PAID", "5002,C02,G2,1,2026-01-15,delivered,6999,CARD,PAID",
```

```
    "5003,C03,G3,2,2026-01-18,shipped,3598,UPI,PAID",   "5004,C05,G4,1,2026-01-22,placed,0,COD,PENDING",
    "5005,C09,G1,1,2026-01-25,delivered,2499,UPI,PAID", "5006,C04,G5,0,2026-01-26,placed,0,UPI,PENDING",
    "5007,C02,G9,1,2026-01-28,delivered,999,CARD,PAID", "5008,C01,G5,3,2026-01-30,delivered,3897,CARD,PAID"])
write_batch("orders_2026-02-01.csv", [                   # daily files that arrive later
    "5004,C05,G4,1,2026-01-22,shipped,3299,UPI,PAID",    # status update of a historical order
    "5009,C03,G2,1,2026-02-01,placed,6999,CARD,PAID", "5010,C04,G1,2,2026-02-01,delivered,4998,UPI,PAID"])
write_batch("orders_2026-02-02.csv", [
    "5011,C01,G3,1,2026-02-02,placed,1799,UPI,PAID", "5012,C05,G2,1,2026-02-02,delivered,6000,CARD,PAID"])
print("Inbox files:", sorted(os.listdir("inbox")))
```

**Output:**

```
Name: Ayushman Rajeev Verma
Inbox files: ['orders_2026-01.csv', 'orders_2026-02-01.csv', 'orders_2026-02-02.csv']
```

## Step 3: ETL Class - Clean, Validate and Load Each File (Historical Load)

The class cleans each file, rejects invalid orders with a reason, upserts orders and payments, and records every processed file in the pipeline_runs table.

**Code:**

```
print("Name: Ayushman Rajeev Verma")
class GadgetHubETL:
    """Extract -> clean -> load, one inbox file at a time. Every processed file is logged, so a file is never loaded
twice."""
    def __init__(self, db_path):
        self.con = sqlite3.connect(db_path)
        self.con.executescript("""
        PRAGMA foreign_keys = ON;
        DROP TABLE IF EXISTS payments; DROP TABLE IF EXISTS orders; DROP TABLE IF EXISTS rejected_orders;
        DROP TABLE IF EXISTS pipeline_runs; DROP TABLE IF EXISTS products; DROP TABLE IF EXISTS customers;
        CREATE TABLE customers (customer_id TEXT PRIMARY KEY, name TEXT, email TEXT, city TEXT);
        CREATE TABLE products  (sku TEXT PRIMARY KEY, product_name TEXT, category TEXT, price REAL);
        CREATE TABLE orders (order_id INTEGER PRIMARY KEY, customer_id TEXT NOT NULL REFERENCES customers, sku TEXT
NOT NULL REFERENCES products,
                             qty INTEGER CHECK (qty > 0), order_date TEXT, status TEXT);
        CREATE TABLE payments (order_id INTEGER PRIMARY KEY REFERENCES orders, amount REAL CHECK (amount > 0), mode
TEXT, status TEXT);
        CREATE TABLE rejected_orders (file_name TEXT, order_id INTEGER, reason TEXT);
        CREATE TABLE pipeline_runs (file_name TEXT PRIMARY KEY, rows_in INTEGER, orders_new INTEGER, orders_updated
INTEGER, payments_saved INTEGER, rejected INTEGER);
        """)

    def load_masters(self, customers, products):
        c = customers.assign(name=customers["name"].str.strip().str.title(), email=customers["email"].str.lower())
        c.loc[~c["email"].str.match(r"^[\w.]+@\w+\.[a-z]{2,}$"), "email"] = None          # invalid e-mail -> NULL
        c.to_sql("customers", self.con, if_exists="append", index=False)
        products.to_sql("products", self.con, if_exists="append", index=False)

    def process_file(self, path):
        name = os.path.basename(path)
        df = pd.read_csv(path)
        df["status"] = df["status"].str.upper()
        customers = set(pd.read_sql("SELECT customer_id FROM customers", self.con)["customer_id"])
        skus = set(pd.read_sql("SELECT sku FROM products", self.con)["sku"])
        reason = pd.Series("", index=df.index)
        reason[df["qty"] <= 0] = "quantity must be positive"
        reason[~df["sku"].isin(skus)] = "unknown product"
        reason[~df["customer_id"].isin(customers)] = "unknown customer"
        bad, good = df[reason != ""], df[reason == ""]
        for oid, why in zip(bad["order_id"], reason[reason != ""]):
            self.con.execute("INSERT INTO rejected_orders VALUES (?,?,?)", (name, int(oid), why))

        existing = set(pd.read_sql("SELECT order_id FROM orders", self.con)["order_id"])
        new = int((~good["order_id"].isin(existing)).sum())
        self.con.executemany("""INSERT INTO orders VALUES (?,?,?,?,?,?)
                                ON CONFLICT(order_id) DO UPDATE SET status = excluded.status""",
                             good[["order_id", "customer_id", "sku", "qty", "order_date",
"status"]].astype(object).values.tolist())
        paid = good[good["pay_amount"] > 0]
        self.con.executemany("""INSERT INTO payments VALUES (?,?,?,?)
                                ON CONFLICT(order_id) DO UPDATE SET amount = excluded.amount, mode = excluded.mode,
status = excluded.status""",
                             paid[["order_id", "pay_amount", "pay_mode",
"pay_status"]].astype(object).values.tolist())
        self.con.execute("INSERT INTO pipeline_runs VALUES (?,?,?,?,?,?)", (name, len(df), new, len(good) - new,
len(paid), len(bad)))
```

```
        self.con.commit()
        print(f"  {name:<22} rows={len(df)}  new={new}  updated={len(good) - new}  payments={len(paid)}
rejected={len(bad)}")

    def run(self, inbox):
        done = {r[0] for r in self.con.execute("SELECT file_name FROM pipeline_runs")}
        todo = [p for p in sorted(glob.glob(f"{inbox}/orders_*.csv")) if os.path.basename(p) not in done]
        print(f"Run: {len(todo)} new file(s) to process, {len(done)} already processed")
        for p in todo:
            self.process_file(p)

etl = GadgetHubETL("gadgethub.db")
etl.load_masters(customers_df, products_df)
print("RUN 1 - historical file + the first daily files")
etl.run("inbox")
print("\nRejected orders:")
print(pd.read_sql("SELECT * FROM rejected_orders", etl.con).to_string(index=False))
```

**Output:**

```
Name: Ayushman Rajeev Verma
RUN 1 - historical file + the first daily files
Run: 3 new file(s) to process, 0 already processed
  orders_2026-01.csv     rows=8  new=5  updated=0  payments=4  rejected=3
  orders_2026-02-01.csv  rows=3  new=2  updated=1  payments=3  rejected=0
  orders_2026-02-02.csv  rows=2  new=2  updated=0  payments=2  rejected=0

Rejected orders:
         file_name  order_id                    reason
orders_2026-01.csv      5005           unknown customer
orders_2026-01.csv      5006 quantity must be positive
orders_2026-01.csv      5007            unknown product
```

## Step 4: Newly Arriving Data - Process Only New Files

**Code:**

```
# NEW DATA ARRIVES: one more daily file, containing a new order and a status update of an old order
print("Name: Ayushman Rajeev Verma")
write_batch("orders_2026-02-03.csv", [
    "5013,C02,G4,2,2026-02-03,placed,6598,CARD,PAID",
    "5003,C03,G3,2,2026-01-18,delivered,3598,UPI,PAID"])

print("Status of order 5003 BEFORE:", etl.con.execute("SELECT status FROM orders WHERE order_id =
5003").fetchone()[0])
print("RUN 2 - only the new file is processed")
etl.run("inbox")
print("Status of order 5003 AFTER :", etl.con.execute("SELECT status FROM orders WHERE order_id =
5003").fetchone()[0])
print("\nRUN 3 - nothing new in the inbox")
etl.run("inbox")

print("\nPipeline log:")
print(pd.read_sql("SELECT * FROM pipeline_runs", etl.con).to_string(index=False))
```

**Output:**

```
Name: Ayushman Rajeev Verma
Status of order 5003 BEFORE: SHIPPED
RUN 2 - only the new file is processed
Run: 1 new file(s) to process, 3 already processed
  orders_2026-02-03.csv  rows=2  new=1  updated=1  payments=2  rejected=0
Status of order 5003 AFTER : DELIVERED

RUN 3 - nothing new in the inbox
Run: 0 new file(s) to process, 4 already processed

Pipeline log:
           file_name  rows_in  orders_new  orders_updated  payments_saved  rejected
   orders_2026-01.csv        8           5               0               4         3
orders_2026-02-01.csv        3           2               1               3         0
orders_2026-02-02.csv        2           2               0               2         0
orders_2026-02-03.csv        2           1               1               2         0
```

## Step 5: Business Report

### Code:

```
print("Name: Ayushman Rajeev Verma")
con = etl.con
print("1. REVENUE BY CATEGORY (cancelled orders excluded)")
print(pd.read_sql("""SELECT p.category, COUNT(*) AS orders, SUM(o.qty) AS units, SUM(o.qty * p.price) AS revenue
                     FROM orders o JOIN products p USING (sku) WHERE o.status <> 'CANCELLED'
                     GROUP BY p.category ORDER BY revenue DESC""", con).to_string(index=False))

print("\n2. DAILY REVENUE WITH RUNNING TOTAL (window function)")
daily = pd.read_sql("""SELECT order_date, SUM(o.qty * p.price) AS revenue,
                              SUM(SUM(o.qty * p.price)) OVER (ORDER BY order_date) AS running_total
                       FROM orders o JOIN products p USING (sku) GROUP BY order_date ORDER BY order_date""", con)
print(daily.to_string(index=False))

print("\n3. PAYMENT CHECK (order value vs amount paid)")
print(pd.read_sql("""SELECT o.order_id, c.name AS customer, o.qty * p.price AS order_value, COALESCE(y.amount, 0) AS
paid,
                            CASE WHEN y.amount IS NULL THEN 'NO PAYMENT'
                                 WHEN y.amount = o.qty * p.price THEN 'MATCHED' ELSE 'MISMATCH' END AS check_result
                     FROM orders o JOIN customers c USING (customer_id) JOIN products p USING (sku)
                     LEFT JOIN payments y USING (order_id) ORDER BY o.order_id""", con).to_string(index=False))

fig, ax = plt.subplots(figsize=(7, 3.2))
ax.bar(daily["order_date"], daily["revenue"], color="#2980b9", label="daily revenue")
ax.plot(daily["order_date"], daily["running_total"], color="#e67e22", marker="o", label="running total")
ax.set_title("GadgetHub revenue"); ax.legend(); plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.gcf().text(0.99, 0.005, "Name: Ayushman Rajeev Verma", ha="right", va="bottom", fontsize=8, color="gray")
plt.savefig("p9_gadgethub.png", dpi=110, bbox_inches="tight")
plt.show()
```

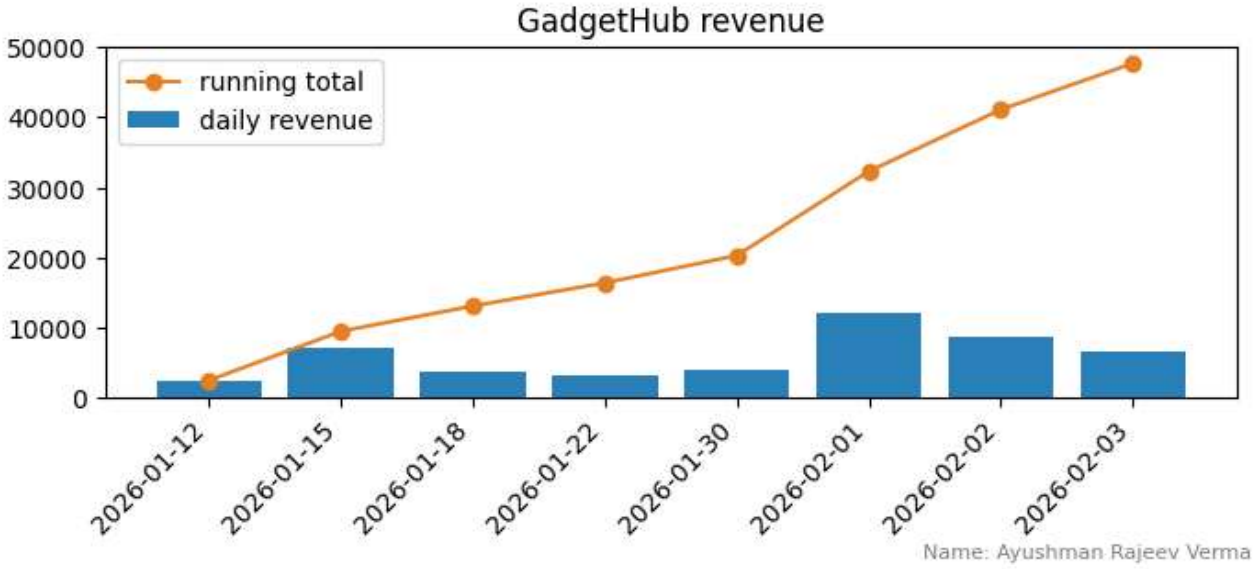
### Output:

```
Name: Ayushman Rajeev Verma
1. REVENUE BY CATEGORY (cancelled orders excluded)
   category  orders  units  revenue
  Wearables       3      3  20997.0
      Audio       4      6  17394.0
Accessories       3      6   9294.0

2. DAILY REVENUE WITH RUNNING TOTAL (window function)
order_date  revenue  running_total
2026-01-12   2499.0         2499.0
2026-01-15   6999.0         9498.0
2026-01-18   3598.0        13096.0
2026-01-22   3299.0        16395.0
2026-01-30   3897.0        20292.0
2026-02-01  11997.0        32289.0
2026-02-02   8798.0        41087.0
2026-02-03   6598.0        47685.0

3. PAYMENT CHECK (order value vs amount paid)
 order_id    customer  order_value   paid check_result
     5001  Aman Gupta       2499.0 2499.0      MATCHED
     5002  Bhavna Rao       6999.0 6999.0      MATCHED
     5003 Chirag Shah       3598.0 3598.0      MATCHED
     5004 Esha Kapoor       3299.0 3299.0      MATCHED
     5008  Aman Gupta       3897.0 3897.0      MATCHED
     5009 Chirag Shah       6999.0 6999.0      MATCHED
     5010 Divya Menon       4998.0 4998.0      MATCHED
     5011  Aman Gupta       1799.0 1799.0      MATCHED
     5012 Esha Kapoor       6999.0 6000.0     MISMATCH
     5013  Bhavna Rao       6598.0 6598.0      MATCHED
```

**Output:**


GadgetHub revenue
running total
daily revenue
50000
40000
30000
20000
10000
0
2026-01-12
2026-01-15
2026-01-18
2026-01-22
2026-01-30
2026-02-01
2026-02-02
2026-02-03
Name: Ayushman Rajeev Verma

# Practical No. 10: Mini Project: Retail Sales Data Warehouse - End-to-End Data Engineering Solution

Aayushman Rajeev Verma | Roll No. 57 | Student ID 5176202
Vedant Vinod Rakshe | Roll No. 37 | Student ID 5176220

**Aim:** To design and implement an end-to-end data engineering solution covering data ingestion, cleaning, transformation, storage, data warehouse design and reporting for a retail sales dataset.

## Step 1: Problem Statement and Solution Architecture

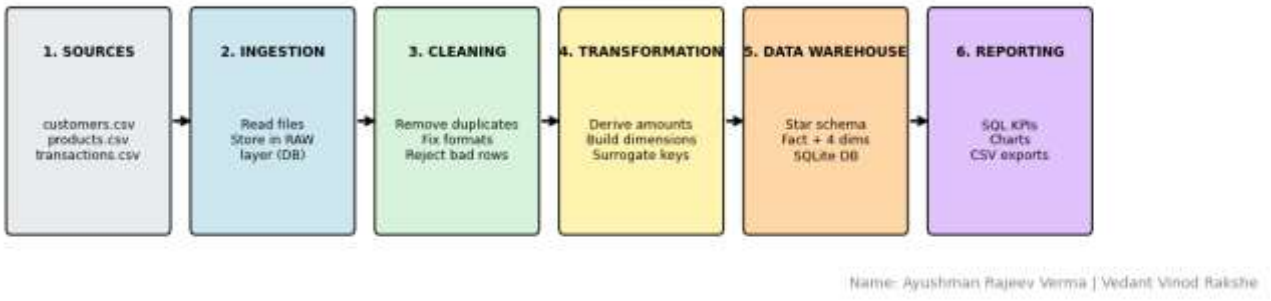
A retail company receives its sales data as raw CSV files (customers, products and transactions). The files contain duplicates, missing values, invalid dates and quantities, unknown products and inconsistent text. The management wants clean, reliable data in a data warehouse so that revenue, product, city and customer reports can be produced with simple SQL queries.

The solution is built as a six-stage pipeline in Python (Pandas, NumPy, Matplotlib) with SQLite as the storage engine. Since the company's real files are not available, the first stage creates a synthetic dataset of 150 customers, 20 products and 1,236 transaction rows (including 36 duplicates) with deliberately injected data-quality problems.

**Code:**

```
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
fig, ax = plt.subplots(figsize=(11, 2.6))
stages = [("1. SOURCES", "customers.csv\nproducts.csv\ntransactions.csv", "#e9ecef"),
          ("2. INGESTION", "Read files\nStore in RAW\nlayer (DB)", "#cde7f0"),
          ("3. CLEANING", "Remove duplicates\nFix formats\nReject bad rows", "#d8f3dc"),
          ("4. TRANSFORMATION", "Derive amounts\nBuild dimensions\nSurrogate keys", "#fff3b0"),
          ("5. DATA WAREHOUSE", "Star schema\nFact + 4 dims\nSQLite DB", "#ffd6a5"),
          ("6. REPORTING", "SQL KPIs\nCharts\nCSV exports", "#e0c3fc")]
for i, (title, body, color) in enumerate(stages):
    x = i * 1.85
    ax.add_patch(FancyBboxPatch((x, 0.2), 1.55, 1.7, boxstyle="round,pad=0.05", fc=color, ec="black"))
    ax.text(x + 0.775, 1.6, title, ha="center", va="center", fontsize=7.5, fontweight="bold")
    ax.text(x + 0.775, 0.9, body, ha="center", va="center", fontsize=7)
    if i < len(stages) - 1:
        ax.annotate("", xy=(x + 1.85, 1.05), xytext=(x + 1.6, 1.05), arrowprops=dict(arrowstyle="->", lw=1.6))
ax.set_xlim(-0.1, 11.2); ax.set_ylim(0, 2.1); ax.axis("off")
plt.gcf().text(0.99, 0.005, "Name: Ayushman Rajeev Verma | Vedant Vinod Rakshe", ha="right", va="bottom",
fontsize=8, color="gray")
plt.savefig("p10_architecture.png", dpi=130, bbox_inches="tight")
```

**Output:**




## Step 2: Source Data Creation and Data Ingestion

**Code:**

```
# Name: Ayushman Rajeev Verma
# Name: Vedant Vinod Rakshe
# MINI PROJECT: Retail Sales Data Warehouse (end-to-end data engineering solution)
print("Name: Ayushman Rajeev Verma | Group partner: Vedant Vinod Rakshe")
import os, sqlite3
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
run_log = []                                          # records every stage of the pipeline
def log(stage, rows_in, rows_out, note=""):
    run_log.append({"stage": stage, "rows_in": rows_in, "rows_out": rows_out, "note": note})

# ------------------------------------------------------------
# STAGE 1 : SOURCE SYSTEM SIMULATION (creates the raw CSV files)
# ------------------------------------------------------------
rng = np.random.default_rng(42)
catalog = {"Electronics": [("Laptop", 55000), ("Smartphone", 22000), ("Headphones", 2500),
                           ("Smart Watch", 6500), ("Mouse", 500), ("Keyboard", 1500)],
           "Furniture":   [("Office Chair", 3500), ("Study Desk", 9000), ("Bookshelf", 6000),
                           ("Table Lamp", 900), ("Sofa", 25000)],
           "Stationery":  [("Notebook", 60), ("Pen Pack", 120), ("Backpack", 1200),
                           ("Calculator", 850), ("Marker Set", 300)],
           "Fashion":     [("T-Shirt", 700), ("Jeans", 1800), ("Sneakers", 3200), ("Jacket", 4500)]}
rows = [(cat, n, p) for cat, items in catalog.items() for n, p in items]
products = pd.DataFrame({"product_id": [f"P{i+1:02d}" for i in range(len(rows))],
                         "product_name": [r[1] for r in rows], "category": [r[0] for r in rows],
                         "unit_price": [r[2] for r in rows]})

first = ["Amit", "Riya", "Neha", "Karan", "Sana", "Dev", "Tina", "Rahul", "Pooja",
         "Vikram", "Anjali", "Rohan"]
last = ["Shah", "Patil", "Rao", "Jain", "Khan", "Mehta", "Dsouza", "Verma", "Kulkarni", "Singh"]
cities = ["Mumbai", "Pune", "Delhi", "Nashik", "Surat", "Goa"]
n_cust = 150
fn, ln = rng.choice(first, n_cust), rng.choice(last, n_cust)
city = rng.choice(cities, n_cust).astype(object)
for i in range(n_cust):                               # messy text formats
    r = rng.random()
    city[i] = city[i].lower() if r < 0.15 else (city[i].upper() + " " if r < 0.25 else city[i])
signup = pd.Timestamp("2025-06-01") + pd.to_timedelta(rng.integers(0, 120, n_cust), unit="D")
customers = pd.DataFrame({
    "customer_id": [f"C{i+1:03d}" for i in range(n_cust)], "name": [f"{a} {b}" for a, b in zip(fn, ln)],
    "email": [f"{a}.{b}{i+1}@Mail.com" for i, (a, b) in enumerate(zip(fn, ln))], "city": city,
    "signup_date": signup.strftime("%Y-%m-%d")})

n_tx = 1200
price_of = dict(zip(products["product_id"], products["unit_price"]))
pid = rng.choice(products["product_id"], n_tx)
qty = rng.integers(1, 6, n_tx)
qty = np.where(np.array([price_of[p] for p in pid]) > 20000, rng.integers(1, 3, n_tx), qty)
days = pd.Timestamp("2025-10-01") + pd.to_timedelta(rng.integers(0, 182, n_tx), unit="D")
tx = pd.DataFrame({
    "order_id": 100001 + np.arange(n_tx),
    "order_date": days.strftime("%Y-%m-%d"),
    "customer_id": rng.choice(customers["customer_id"], n_tx), "product_id": pid, "quantity": qty,
    "discount_pct": rng.choice([0, 0, 0, 5, 10, 15], n_tx),
    "payment_mode": rng.choice(["UPI", "CARD", "COD", "NETBANKING"], n_tx, p=[0.4, 0.3, 0.15, 0.15])})

# inject data-quality problems into the transactions
bad = rng.choice(n_tx, 72, replace=False)
tx.loc[bad[:24], "customer_id"] = None                # missing customer
tx.loc[bad[24:42], "quantity"] = rng.choice([0, -1], 18) # invalid quantity
tx.loc[bad[42:60], "order_date"] = rng.choice(["N/A", "31-02-2026"], 18)   # invalid date
tx.loc[bad[60:72], "product_id"] = "P99"              # unknown product
tx["payment_mode"] = [m.lower() if r < 0.1 else (m.title() if r < 0.2 else m)
                      for m, r in zip(tx["payment_mode"], rng.random(n_tx))]
tx = pd.concat([tx, tx.sample(36, random_state=1)], ignore_index=True)    # 36 duplicate rows

os.makedirs("source", exist_ok=True)
products.to_csv("source/products.csv", index=False)
customers.to_csv("source/customers.csv", index=False)
tx.to_csv("source/transactions.csv", index=False)

# ------------------------------------------------------------
# STAGE 2 : DATA INGESTION  (read source files, store in RAW layer)
# ------------------------------------------------------------
if os.path.exists("warehouse.db"):
    os.remove("warehouse.db")
conn = sqlite3.connect("warehouse.db")
raw = {name: pd.read_csv(f"source/{name}.csv") for name in ["customers", "products", "transactions"]}
for name, df in raw.items():
    df.assign(batch_id="BATCH_001").to_sql(f"raw_{name}", conn, if_exists="replace", index=False)
    log("ingest_" + name, len(df), len(df))
    print(f"Ingested {name:<13}: {len(df):>5} rows, {df.shape[1]} columns -> table raw_{name}")
print("\nSample of raw transactions:")
print(raw["transactions"].head(5).to_string(index=False))
```

**Output:**

```
Name: Ayushman Rajeev Verma | Group partner: Vedant Vinod Rakshe
Ingested customers    :   150 rows, 5 columns -> table raw_customers
Ingested products     :    20 rows, 4 columns -> table raw_products
Ingested transactions :  1236 rows, 7 columns -> table raw_transactions

Sample of raw transactions:
 order_id order_date customer_id product_id  quantity  discount_pct payment_mode
   100001 2026-02-04        C094        P01         2             0   NETBANKING
   100002 2026-01-06        C058        P01         1            10         card
   100003 2026-03-31        C042        P05         1            10         CARD
   100004 2026-02-18        C065        P18         1             5         CARD
   100005 2025-10-04        C047        P10         5             5          COD
```

## Step 3: Data Cleaning and Data Quality Report

Five validation rules are applied to the transactions: remove duplicates, invalid dates, invalid quantities, missing customer ids and unknown products. Payment modes and customer text fields are standardised.

**Code:**

```
# ------------------------------------------------------------
# STAGE 3 : DATA CLEANING  (with a data-quality report)
# ------------------------------------------------------------
print("Name: Ayushman Rajeev Verma | Group partner: Vedant Vinod Rakshe")
t = raw["transactions"].copy()
dq = [("Raw rows received", len(t))]

n = len(t); t = t.drop_duplicates()
dq.append(("Duplicate rows removed", n - len(t)))

t["order_date"] = pd.to_datetime(t["order_date"], format="%Y-%m-%d", errors="coerce")
n = len(t); t = t[t["order_date"].notna()]
dq.append(("Invalid dates removed", n - len(t)))

n = len(t); t = t[t["quantity"] > 0]
dq.append(("Invalid quantity removed", n - len(t)))

n = len(t); t = t[t["customer_id"].notna()]
dq.append(("Missing customer_id removed", n - len(t)))

n = len(t); t = t[t["product_id"].isin(raw["products"]["product_id"])]
dq.append(("Unknown product removed", n - len(t)))

t["payment_mode"] = t["payment_mode"].str.strip().str.upper()        # UPI / Upi / upi -> UPI
dq.append(("Clean rows kept", len(t)))
clean_tx = t.reset_index(drop=True)

c = raw["customers"].copy()
c["name"] = c["name"].str.strip().str.title()
c["city"] = c["city"].str.strip().str.title()                         # 'PUNE ' -> 'Pune'
c["email"] = c["email"].str.strip().str.lower()
clean_cust = c.drop_duplicates("customer_id")
clean_prod = raw["products"].drop_duplicates("product_id")

print(pd.DataFrame(dq, columns=["Data quality check", "Rows"]).to_string(index=False))
print("\nCity values before:", sorted(raw["customers"]["city"].unique())[:6], "...")
print("City values after :", sorted(clean_cust["city"].unique()))
log("clean_transactions", len(raw["transactions"]), len(clean_tx), "5 quality rules applied")
log("clean_customers", len(raw["customers"]), len(clean_cust), "text standardised")
```

**Output:**

```
Name: Ayushman Rajeev Verma | Group partner: Vedant Vinod Rakshe
         Data quality check  Rows
          Raw rows received  1236
     Duplicate rows removed    36
      Invalid dates removed    18
   Invalid quantity removed    18
Missing customer_id removed    24
    Unknown product removed    12
            Clean rows kept  1128

City values before: ['DELHI ', 'Delhi', 'GOA ', 'Goa', 'MUMBAI ', 'Mumbai'] ...
City values after : ['Delhi', 'Goa', 'Mumbai', 'Nashik', 'Pune', 'Surat']
```

## Step 4: Data Transformation

Gross, discount and net amounts are derived, and the clean data is reshaped into dimension tables (date, customer, product, payment) and a fact table that uses surrogate keys.

**Code:**

```
# ------------------------------------------------------------
# STAGE 4 : TRANSFORMATION  (business columns + dimension tables)
# ------------------------------------------------------------
# Derived measures
print("Name: Ayushman Rajeev Verma | Group partner: Vedant Vinod Rakshe")
f = clean_tx.merge(clean_prod[["product_id", "unit_price"]], on="product_id")
f["gross_amount"] = f["quantity"] * f["unit_price"]
f["discount_amount"] = (f["gross_amount"] * f["discount_pct"] / 100).round(2)
f["net_amount"] = f["gross_amount"] - f["discount_amount"]

# Dimension tables with surrogate keys
dim_date = pd.DataFrame({"full_date": pd.date_range(f["order_date"].min(), f["order_date"].max())})
dim_date["date_key"] = dim_date["full_date"].dt.strftime("%Y%m%d").astype(int)
dim_date["day_name"] = dim_date["full_date"].dt.day_name()
dim_date["month_name"] = dim_date["full_date"].dt.strftime("%b %Y")
dim_date["month_num"] = dim_date["full_date"].dt.strftime("%Y-%m")
dim_date["quarter"] = "Q" + dim_date["full_date"].dt.quarter.astype(str)
dim_date["year"] = dim_date["full_date"].dt.year
dim_date["is_weekend"] = dim_date["full_date"].dt.dayofweek.isin([5, 6]).astype(int)
dim_date["full_date"] = dim_date["full_date"].dt.strftime("%Y-%m-%d")
dim_date = dim_date[["date_key", "full_date", "day_name", "month_name", "month_num",
                     "quarter", "year", "is_weekend"]]

dim_customer = clean_cust.reset_index(drop=True)
dim_customer.insert(0, "customer_key", dim_customer.index + 1)
dim_product = clean_prod.reset_index(drop=True)
dim_product.insert(0, "product_key", dim_product.index + 1)
dim_payment = pd.DataFrame({"payment_mode": sorted(f["payment_mode"].unique())})
dim_payment.insert(0, "payment_key", dim_payment.index + 1)

# Fact table: replace business keys with surrogate keys
fact_sales = (f.assign(date_key=f["order_date"].dt.strftime("%Y%m%d").astype(int))
               .merge(dim_customer[["customer_key", "customer_id"]], on="customer_id")
               .merge(dim_product[["product_key", "product_id"]], on="product_id")
               .merge(dim_payment, on="payment_mode"))
fact_sales = fact_sales[["order_id", "date_key", "customer_key", "product_key", "payment_key", "quantity",
                         "unit_price", "discount_pct", "gross_amount", "discount_amount", "net_amount"]]
fact_sales.insert(0, "sales_key", range(1, len(fact_sales) + 1))

print("fact_sales rows :", len(fact_sales), "| dim_date:", len(dim_date), "| dim_customer:",
      len(dim_customer), "| dim_product:", len(dim_product), "| dim_payment:", len(dim_payment))
print("\nFact table sample:")
show = ["sales_key", "order_id", "date_key", "customer_key", "product_key", "quantity", "net_amount"]
print(fact_sales[show].head(5).to_string(index=False))
log("transform", len(clean_tx), len(fact_sales), "fact + 4 dimensions built")
```

**Output:**

```
Name: Ayushman Rajeev Verma | Group partner: Vedant Vinod Rakshe
fact_sales rows : 1128 | dim_date: 182 | dim_customer: 150 | dim_product: 20 | dim_payment: 4

Fact table sample:
 sales_key  order_id  date_key  customer_key  product_key  quantity  net_amount
         1    100001  20260204            94            1         2    110000.0
         2    100002  20260106            58            1         1     49500.0
         3    100003  20260331            42            5         1       450.0
         4    100004  20260218            65           18         1      1710.0
         5    100005  20251004            47           10         5      4275.0
```

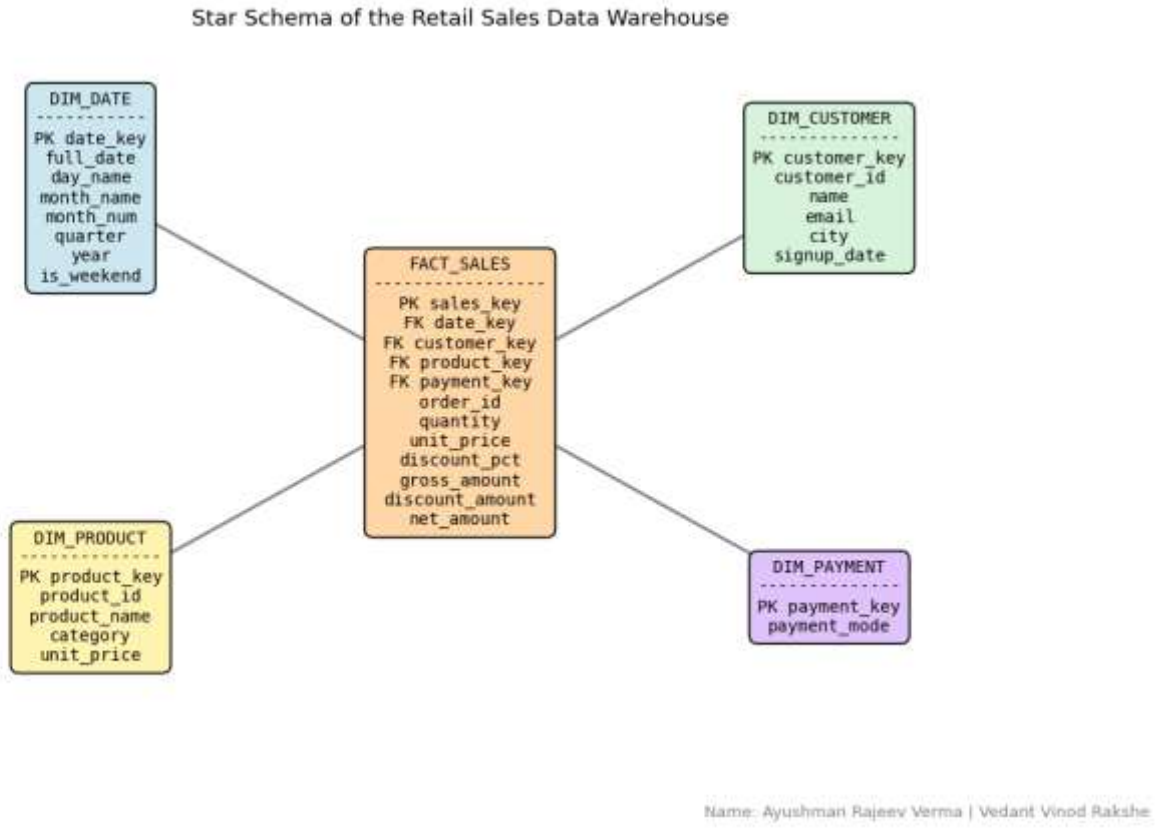
## Step 5: Data Warehouse Design (Star Schema), Storage and Load

A star schema was chosen because it makes analytical queries simple and fast: one central fact table holds the measurable events (sales) and four dimension tables describe them.

**Code:**

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(10, 6.2))
def box(x, y, text, color):
    ax.text(x, y, text, ha="center", va="center", family="monospace", fontsize=8.5,
            bbox=dict(boxstyle="round,pad=0.5", fc=color, ec="black"), zorder=3)
centers = {"date": (1.7, 5.0), "cust": (8.3, 5.0), "prod": (1.7, 1.2), "pay": (8.3, 1.2)}
for (x, y) in centers.values():
    ax.plot([5, x], [3.1, y], color="gray", lw=1.5, zorder=1)
box(5, 3.1, "FACT_SALES\n-----------------\nPK sales_key\nFK date_key\nFK customer_key\nFK product_key\nFK
payment_key\norder_id\nquantity\nunit_price\ndiscount_pct\ngross_amount\ndiscount_amount\nnet_amount", "#ffd6a5")
box(*centers["date"], "DIM_DATE\n-----------\nPK
date_key\nfull_date\nday_name\nmonth_name\nmonth_num\nquarter\nyear\nis_weekend", "#cde7f0")
box(*centers["cust"], "DIM_CUSTOMER\n--------------\nPK customer_key\ncustomer_id\nname\nemail\ncity\nsignup_date",
"#d8f3dc")
box(*centers["prod"], "DIM_PRODUCT\n--------------\nPK product_key\nproduct_id\nproduct_name\ncategory\nunit_price",
"#fff3b0")
box(*centers["pay"], "DIM_PAYMENT\n--------------\nPK payment_key\npayment_mode", "#e0c3fc")
ax.set_xlim(0, 10); ax.set_ylim(0, 6.4); ax.axis("off")
ax.set_title("Star Schema of the Retail Sales Data Warehouse", fontsize=11)
plt.gcf().text(0.99, 0.005, "Name: Ayushman Rajeev Verma | Vedant Vinod Rakshe", ha="right", va="bottom",
fontsize=8, color="gray")
plt.savefig("p10_star_schema.png", dpi=120, bbox_inches="tight")
```

**Output:**




**Code:**

```
# ------------------------------------------------------------------
# STAGE 5 : STORAGE - DATA WAREHOUSE DESIGN (star schema) AND LOAD
# ------------------------------------------------------------------
print("Name: Ayushman Rajeev Verma | Group partner: Vedant Vinod Rakshe")
conn.execute("PRAGMA foreign_keys = ON")
conn.executescript("""
CREATE TABLE dim_date (date_key INTEGER PRIMARY KEY, full_date TEXT, day_name TEXT, month_name TEXT,
                       month_num TEXT, quarter TEXT, year INTEGER, is_weekend INTEGER);
CREATE TABLE dim_customer (customer_key INTEGER PRIMARY KEY, customer_id TEXT UNIQUE, name TEXT,
                           email TEXT, city TEXT, signup_date TEXT);
CREATE TABLE dim_product (product_key INTEGER PRIMARY KEY, product_id TEXT UNIQUE, product_name TEXT,
                          category TEXT, unit_price REAL);
CREATE TABLE dim_payment (payment_key INTEGER PRIMARY KEY, payment_mode TEXT UNIQUE);
CREATE TABLE fact_sales (
    sales_key INTEGER PRIMARY KEY, order_id INTEGER UNIQUE,
    date_key INTEGER REFERENCES dim_date(date_key),
    customer_key INTEGER REFERENCES dim_customer(customer_key),
    product_key INTEGER REFERENCES dim_product(product_key),
    payment_key INTEGER REFERENCES dim_payment(payment_key),
    quantity INTEGER CHECK (quantity > 0), unit_price REAL, discount_pct REAL,
    gross_amount REAL, discount_amount REAL, net_amount REAL);
CREATE INDEX idx_fact_date ON fact_sales(date_key);
CREATE INDEX idx_fact_customer ON fact_sales(customer_key);
CREATE INDEX idx_fact_product ON fact_sales(product_key);
""")
for name, df in [("dim_date", dim_date), ("dim_customer", dim_customer), ("dim_product", dim_product),
                 ("dim_payment", dim_payment), ("fact_sales", fact_sales)]:
    df.to_sql(name, conn, if_exists="append", index=False)
    print(f"{name:<13} loaded: {conn.execute(f'SELECT COUNT(*) FROM {name}').fetchone()[0]:>5} rows")
conn.commit()

violations = conn.execute("PRAGMA foreign_key_check").fetchall()
print("\nForeign key violations:", len(violations))
print("Revenue check -> Pandas:", round(fact_sales["net_amount"].sum(), 2),
      "| SQL:", round(conn.execute("SELECT SUM(net_amount) FROM fact_sales").fetchone()[0], 2))
n_fact = conn.execute("SELECT COUNT(*) FROM fact_sales").fetchone()[0]
log("load_warehouse", len(fact_sales), n_fact, "FK check passed")
```

**Output:**

```
Name: Ayushman Rajeev Verma | Group partner: Vedant Vinod Rakshe
dim_date      loaded:   182 rows
dim_customer  loaded:   150 rows
dim_product   loaded:    20 rows
dim_payment   loaded:     4 rows
fact_sales    loaded:  1128 rows

Foreign key violations: 0
Revenue check -> Pandas: 15239825.5 | SQL: 15239825.5
```

## Step 6: Reporting and Analytics

**Code:**

```
# ------------------------------------------------------------
# STAGE 6 : REPORTING  (SQL on the star schema)
# ------------------------------------------------------------
print("Name: Ayushman Rajeev Verma | Group partner: Vedant Vinod Rakshe")
q = lambda sql: pd.read_sql(sql, conn)

kpi = q("""SELECT COUNT(*) AS orders, SUM(quantity) AS units,
                  ROUND(SUM(net_amount), 2) AS net_revenue,
                  ROUND(SUM(discount_amount), 2) AS discount_given,
                  ROUND(AVG(net_amount), 2) AS avg_order_value
           FROM fact_sales""")
print("KEY PERFORMANCE INDICATORS"); print(kpi.to_string(index=False))

monthly = q("""SELECT d.month_num AS month, COUNT(*) AS orders, ROUND(SUM(f.net_amount), 2) AS net_revenue
               FROM fact_sales f JOIN dim_date d ON d.date_key = f.date_key
               GROUP BY d.month_num ORDER BY d.month_num""")
print("\nMONTHLY REVENUE"); print(monthly.to_string(index=False))

by_cat = q("""SELECT p.category, SUM(f.quantity) AS units, ROUND(SUM(f.net_amount), 2) AS net_revenue
              FROM fact_sales f JOIN dim_product p ON p.product_key = f.product_key
              GROUP BY p.category ORDER BY net_revenue DESC""")
print("\nREVENUE BY CATEGORY"); print(by_cat.to_string(index=False))

by_city = q("""SELECT c.city, COUNT(*) AS orders, ROUND(SUM(f.net_amount), 2) AS net_revenue
               FROM fact_sales f JOIN dim_customer c ON c.customer_key = f.customer_key
               GROUP BY c.city ORDER BY net_revenue DESC""")
print("\nREVENUE BY CITY"); print(by_city.to_string(index=False))

top_products = q("""SELECT p.product_name, p.category, SUM(f.quantity) AS units,
                           ROUND(SUM(f.net_amount), 2) AS net_revenue
                    FROM fact_sales f JOIN dim_product p ON p.product_key = f.product_key
                    GROUP BY p.product_name ORDER BY net_revenue DESC LIMIT 5""")
print("\nTOP 5 PRODUCTS"); print(top_products.to_string(index=False))

top_customers = q("""SELECT c.name, c.city, COUNT(*) AS orders,
                            ROUND(SUM(f.net_amount), 2) AS net_revenue
                     FROM fact_sales f JOIN dim_customer c ON c.customer_key = f.customer_key
                     GROUP BY c.customer_key ORDER BY net_revenue DESC LIMIT 5""")
print("\nTOP 5 CUSTOMERS"); print(top_customers.to_string(index=False))

pay = q("""SELECT y.payment_mode, COUNT(*) AS orders, ROUND(SUM(f.net_amount), 2) AS net_revenue
           FROM fact_sales f JOIN dim_payment y ON y.payment_key = f.payment_key
           GROUP BY y.payment_mode ORDER BY orders DESC""")
print("\nPAYMENT MODES"); print(pay.to_string(index=False))

weekend = q("""SELECT CASE d.is_weekend WHEN 1 THEN 'Weekend' ELSE 'Weekday' END AS day_type,
                      COUNT(*) AS orders, ROUND(AVG(f.net_amount), 2) AS avg_order_value
               FROM fact_sales f JOIN dim_date d ON d.date_key = f.date_key GROUP BY d.is_weekend""")
print("\nWEEKDAY vs WEEKEND"); print(weekend.to_string(index=False))
log("reporting", len(fact_sales), 8, "8 report queries")
```

**Output:**

```
Name: Ayushman Rajeev Verma | Group partner: Vedant Vinod Rakshe
KEY PERFORMANCE INDICATORS
 orders  units  net_revenue  discount_given  avg_order_value
   1128   3112   15239825.5        816984.5         13510.48

MONTHLY REVENUE
  month  orders  net_revenue
2025-10     188    2607159.5
2025-11     189    2220079.0
2025-12     177    2802472.0
2026-01     198    3123208.0
2026-02     189    2366325.0
2026-03     187    2120582.0

REVENUE BY CATEGORY
   category  units  net_revenue
Electronics    820    7548200.0
  Furniture    837    5600050.0
    Fashion    680    1729335.0
 Stationery    775     362240.5

REVENUE BY CITY
  city  orders  net_revenue
Mumbai     244    3436139.5
  Pune     248    3282973.5
   Goa     208    2661399.0
 Surat     165    2590624.0
Nashik     149    2185402.5
 Delhi     114    1083287.0

TOP 5 PRODUCTS
product_name    category  units  net_revenue
      Laptop Electronics     81    4213000.0
        Sofa   Furniture     93    2188750.0
  Smartphone Electronics     76    1557600.0
  Study Desk   Furniture    167    1438650.0
   Bookshelf   Furniture    202    1166100.0

TOP 5 CUSTOMERS
         name   city  orders  net_revenue
   Tina Singh Mumbai      13     358690.0
    Sana Shah Mumbai      15     355644.0
   Pooja Jain   Pune      12     290610.0
Anjali Dsouza Mumbai      12     286442.0
   Amit Mehta    Goa      12     267570.0

PAYMENT MODES
payment_mode  orders  net_revenue
         UPI     473    6430768.5
        CARD     351    4433682.5
         COD     154    2338783.0
  NETBANKING     150    2036591.5

WEEKDAY vs WEEKEND
day_type  orders  avg_order_value
 Weekday     812         13560.27
 Weekend     316         13382.55
```

## Step 7: Dashboard, Report Export and Pipeline Run Summary

**Code:**

```
# ---- Dashboard charts and report exports
print("Name: Ayushman Rajeev Verma | Group partner: Vedant Vinod Rakshe")
fig, ax = plt.subplots(2, 2, figsize=(11, 7))
ax[0, 0].plot(monthly["month"], monthly["net_revenue"], marker="o", color="#2a6f97")
ax[0, 0].set_title("Monthly Net Revenue"); ax[0, 0].tick_params(axis="x", rotation=30)
ax[0, 1].bar(by_cat["category"], by_cat["net_revenue"], color="#52b788")
ax[0, 1].set_title("Net Revenue by Category")
ax[1, 0].barh(by_city["city"], by_city["net_revenue"], color="#f4a261")
```

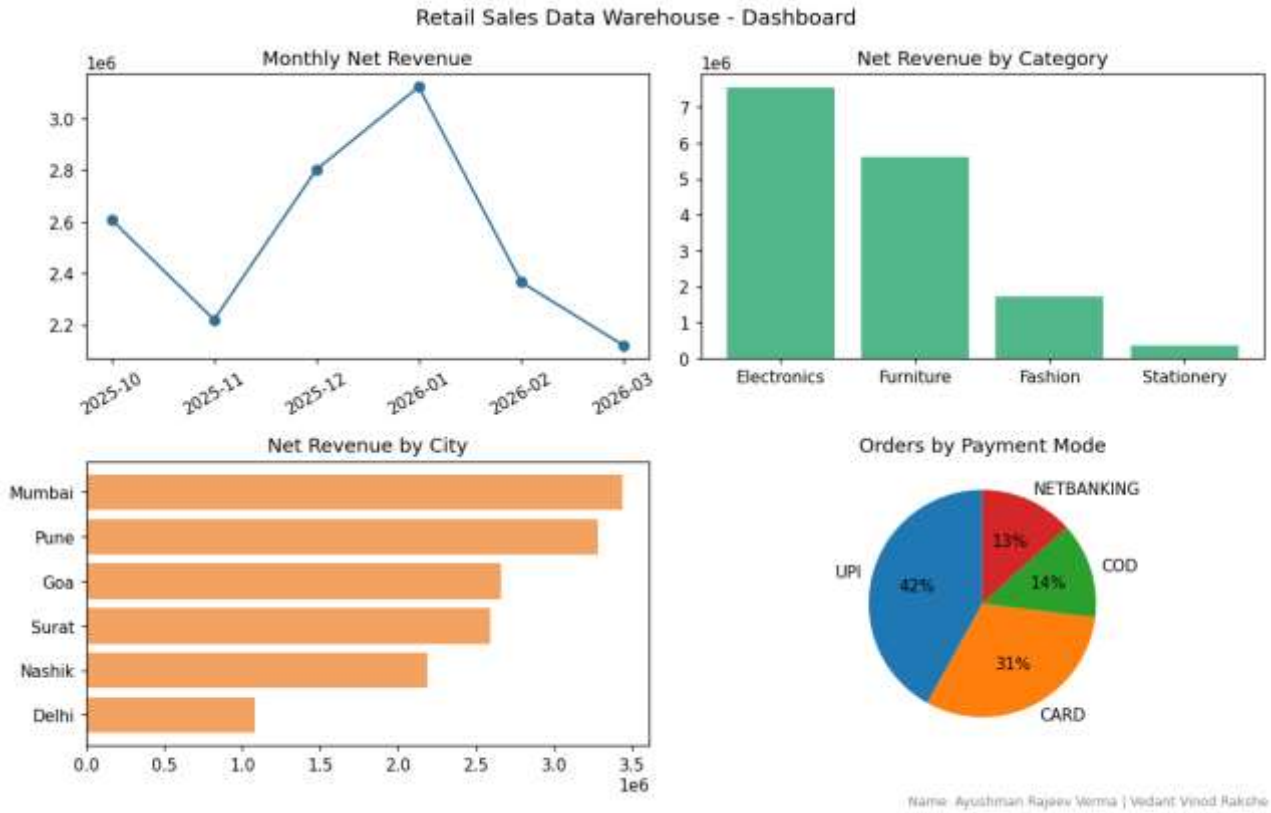
```
ax[1, 0].set_title("Net Revenue by City"); ax[1, 0].invert_yaxis()
ax[1, 1].pie(pay["orders"], labels=pay["payment_mode"], autopct="%1.0f%%", startangle=90)
ax[1, 1].set_title("Orders by Payment Mode")
plt.suptitle("Retail Sales Data Warehouse - Dashboard", fontsize=13)
plt.tight_layout()
plt.gcf().text(0.99, 0.005, "Name: Ayushman Rajeev Verma | Vedant Vinod Rakshe", ha="right", va="bottom",
fontsize=8, color="gray")
plt.savefig("p10_dashboard.png", dpi=110, bbox_inches="tight")
plt.show()

os.makedirs("reports", exist_ok=True)
for name, df in [("kpi", kpi), ("monthly_revenue", monthly), ("by_category", by_cat),
                 ("by_city", by_city), ("top_products", top_products), ("top_customers", top_customers)]:
    df.to_csv(f"reports/{name}.csv", index=False)
print("Reports exported to the reports/ folder:")
for file in sorted(os.listdir("reports")):
    print("  ", file)
```

**Output:**

```
Name: Ayushman Rajeev Verma | Group partner: Vedant Vinod Rakshe
Reports exported to the reports/ folder:
   by_category.csv
   by_city.csv
   kpi.csv
   monthly_revenue.csv
   top_customers.csv
   top_products.csv
```

**Output:**




**Code:**

```
# ---- Pipeline run summary (audit trail of all stages)
print("Name: Ayushman Rajeev Verma | Group partner: Vedant Vinod Rakshe")
print(pd.DataFrame(run_log).to_string(index=False))
conn.close()
```

**Output:**

```
Name: Ayushman Rajeev Verma | Group partner: Vedant Vinod Rakshe
              stage  rows_in  rows_out                      note
   ingest_customers      150       150
    ingest_products       20        20
ingest_transactions     1236      1236
 clean_transactions     1236      1128   5 quality rules applied
    clean_customers      150       150         text standardised
          transform     1128      1128 fact + 4 dimensions built
     load_warehouse     1128      1128           FK check passed
          reporting     1128         8          8 report queries
```